



FACULTY OF INFORMATION TECHNOLOGY AND ELECTRICAL ENGINEERING
DEGREE PROGRAMME IN ELECTRONICS AND COMMUNICATIONS ENGINEERING

MASTER'S THESIS

LIDAR AIDED DYNAMIC DIGITAL TWINS

Author	Gihan Wansekara
Supervisor	Prof. Nandana Rajatheva
Second Examiner	Dr. Himal A. Suraweera
Technical Advisor	Dileepa Marasinghe

June 2025

Wansekara G. (2025) Lidar Aided Dynamic Digital Twins. University of Oulu, Faculty of Information Technology and Electrical Engineering, Degree Programme in Electronics and Communications Engineering. Master's Thesis, 56 p.

ABSTRACT

The continuous advancement of wireless communication technologies, especially the transition towards sixth generation (6G) networks, demands new methodologies for network planning, optimization, and intelligent resource allocation. The increasing complexity of wireless environments, characterized by dynamic obstacles, real-time user mobility, and varying signal propagation conditions, necessitates the integration of artificial intelligence (AI), machine learning (ML), and digital twin (DT) technologies. Digital twins provide an advanced approach by creating high-fidelity virtual replicas of physical networks, enabling extensive performance evaluation, real-time monitoring, and predictive network optimization before real-world deployment.

This research presents a digital twin based simulation framework that models dynamic physical environments using NVIDIA Isaac Sim which is a powerful simulation platform designed for AI-based robotics and environmental modeling. By leveraging LiDAR sensor data, the framework captures real time environmental dynamics, such as structural changes, object movements, and urban variations, and integrates these into NVIDIA Sienna, which is a ray tracing engine for wireless communication simulation. The machine learning based processing pipeline extracts critical environmental features and embeds them into the ray tracing environment to enhance the accuracy of wireless signal propagation modeling.

By incorporating ML driven techniques, the proposed digital twin framework provides a virtual testbed for AI powered radio access networks (AI-RAN), allowing for real time network adaptation, predictive analysis, and intelligent decision making. The developed system enables wireless communication researchers and engineers to simulate, evaluate, and optimize 6G networks efficiently while reducing the reliance on costly and time-consuming real-world experimentation.

The results of this study demonstrate that ML enhanced, digital twin based simulations significantly improve network adaptability, signal prediction accuracy, and energy efficiency. The proposed framework serves as a scalable, AI powered solution for optimizing next-generation wireless networks, enabling future 6G systems to handle the increasing demands of ultra reliable, high speed and low latency communication in complex, real world environments.

Keywords: Digital twin, 6G networks, AI-RAN, machine learning, LiDAR, ray tracing, wireless communication, network optimization, simulation environment, artificial intelligence, real time adaptation, smart resource management, signal propagation.

Wansekara G. (2025) Lidar-avusteiset dynaamiset digitaaliset kaksoiset. Oulun yliopisto, tieto- ja sähkötekniikan tiedekunta, elektroniikan ja tietoliikennetekniikan tutkinto-ohjelma. Diplomityö, 56 s.

TIIVISTELMÄ

Langattomien viestintätekniikoiden jatkuva kehittyminen, erityisesti siirtyminen kuudennen sukupolven (6G) verkkoihin, vaatii uusia menetelmiä verkon suunnitteluun, optimointiin ja älykkääseen resurssien allokointiin. Langattomien ympäristöjen lisääntyvä monimutkaisuus, joille ovat ominaisia dynaamiset esteet, reaaliaikainen käyttäjien liikkuvuus ja vaihtelevat signaalin etenemisolosuhteet, edellyttävät tekoälyn (AI), koneoppimisen (ML) ja digitaalisen kaksoisteknologian (DT) integrointia. Digitaaliset kaksoiset tarjoavat edistyneen lähestymistavan luomalla korkean tarkkuuden virtuaalisia kopioita fyysisistä verkoista, mikä mahdollistaa laajan suorituskyvyn arvioinnin, reaaliaikaisen seurannan ja ennakoivan verkon optimoinnin ennen todellista käyttöönottoa.

Tämä tutkimus esittelee digitaalisen kaksoissimulaatiokehityksen, joka mallintaa dynaamisia fyysisiä ympäristöjä käyttämällä NVIDIA Isaac Simia, joka on tehokas simulointialusta, joka on suunniteltu tekoälypohjaiseen robotiikkaan ja ympäristön mallintamiseen. Hyödyntämällä LiDAR-anturidataa, kehys tallentaa reaaliaikaisen ympäristön dynamiikan, kuten rakenteelliset muutokset, objektien liikkeet ja kaupunkivaihtelut, ja integroi nämä NVIDIA Sionnaan, joka on langattoman viestinnän simulointiin tarkoitettu säteenseurantamoottori. Koneoppimiseen perustuva käsittelyputki poimii kriittiset ympäristöominaisuudet ja upottaa ne säteenseurantaympäristöön parantaakseen langattoman signaalin etenemisen mallinnuksen tarkkuutta.

Sisällyttämällä ML-ohjattuja tekniikoita ehdotettu digitaalinen kaksoiskehys tarjoaa virtuaalisen testialustan tekoälyllä toimiville radioliityntäverkoille (AI-RAN), mikä mahdollistaa reaaliaikaisen verkon mukauttamisen, ennakoivan analyysin ja älykkään päätöksenteon. Kehitetty järjestelmä antaa langattoman viestinnän tutkijoille ja insinööreille mahdollisuuden simuloida, arvioida ja optimoida 6G-verkkoja tehokkaasti samalla, kun se vähentää riippuvuutta kalliista ja aikaa vievästä tosielämän kokeilusta.

Tämän tutkimuksen tulokset osoittavat, että ML-parannetut, digitaaliset kaksoissimulaatiot parantavat merkittävästi verkon mukautumiskykyä, signaalien ennustetarkkuutta ja energiatehokkuutta. Ehdotettu kehys toimii skaalautuvana tekoälypohjaisena ratkaisuna seuraavan sukupolven langattomien verkkojen optimointiin, mikä mahdollistaa tulevien 6G-järjestelmien täyttävän erittäin luotettavan, nopean ja alhaisen latenssin viestinnän kasvavat vaatimukset monimutkaisissa todellisen maailman ympäristöissä.

Avainsanat: Digitaalinen kaksoisverkko, 6G-verkot, AI-RAN, koneoppiminen, LiDAR, säteenseuranta, langaton viestintä, verkon optimointi, simulointiympäristö, tekoäly, reaaliaikainen sopeutuminen, älykäs resurssienhallinta, signaalin eteneminen.

TABLE OF CONTENTS

ABSTRACT

TABLE OF CONTENTS

FOREWORD

LIST OF ABBREVIATIONS AND SYMBOLS

1	INTRODUCTION	9
1.1	Existing work	10
1.2	Thesis contribution	11
1.3	Thesis outline	11
2	THEORETICAL BACKGROUND.....	12
2.1	Digital twin.....	12
2.2	Light detection and ranging (LiDAR)	12
2.2.1	Principles of LiDAR sensing	13
2.2.2	Angular resolution and field of view	13
2.2.3	Intensity and reflectivity	14
2.2.4	LiDAR sensing process	14
2.3	Point cloud generation.....	14
2.4	NVIDIA Isaac Sim	15
2.5	ROS2 for data communication in digital twin frameworks	16
2.6	Clustering and dynamic object detection using DBSCAN	16
2.6.1	DBSCAN algorithm	17
2.7	Fundamentals of ray tracing	18
2.7.1	Far field assumption of a transmit antenna.....	19
2.7.2	Reflection and refraction	19
2.7.3	Fresnel coefficients.....	20
2.7.4	Diffraction	20
2.7.5	Scattering	20
2.7.6	Path loss and phase calculation	21
2.7.7	Ray launching and propagation	21
2.8	NVIDIA Sionna ray tracing	22
3	RELATED WORK	23
3.1	LiDAR aided human blockage prediction for 6G	23
3.2	LoS detection and prediction based on static maps.....	23
3.3	Ray tracing based radio channel modelling applied to RIS	24
3.4	A ray launching approach for computing exact paths with point clouds	25
3.5	Differentiable ray tracing for radio propagation modeling	25
4	IMPLEMENTATION.....	27
4.1	Workflow for real time dynamic digital twins	27
4.2	Scene creation and sensor simulation on pre-developed 3D models in Isaac sim	28
4.2.1	Human dynamic simulation in Isaac Sim	29
4.2.2	Sensor modelling in Isaac Sim	29
4.3	Scene creation and sensor simulation for outdoor environments.....	33

4.4	Dynamics detection using LiDAR ROS2 topics	35
4.5	Emulating dynamics on Sionna and ray tracing.....	36
4.5.1	Scene creation for Sionna	36
4.5.2	TX and RX configuration and placement.....	38
4.5.3	Mobility of the detected dynamic objects	40
4.5.4	Ray tracing and coverage map generation.....	40
4.5.5	Access point selection based on received signal strength	41
4.6	NimbusRT based ray tracing on point clouds	42
5	RESULTS AND DISCUSSION	43
5.1	Human dynamic detection.....	43
5.1.1	Accuracy check of human tracking	44
5.2	Received Signal Strength variations.....	46
5.2.1	RSS with different frequency settings.	46
5.2.2	RSS with different human dynamics	47
5.2.3	NimbusRT based ray tracing on point clouds	49
5.2.4	Practical implementation in a physical environment using real sensors ...	51
6	SUMMARY	54
7	REFERENCES	55

FOREWORD

This research and master's thesis titled 'Lidar Aided Dynamic Digital Twins' was conducted at the Center for Wireless Communications (CWC) at the University of Oulu, Finland. The work was carried out under the AI-RAN research group within the 6G Flagship program.

I would like to express my sincere gratitude to my supervisors, Prof. Nandana Rajatheva from the University of Oulu and Dr. Himal Suraweera from the University of Peradeniya, for the support and guidance given to me. I would also like to extend my heartfelt gratitude to Dr. Dileepa Marasinghe, AI-RAN research manager and technical advisor, for his immense support throughout the research work.

Further, I am grateful to the academic and technical staff at CWC for providing the necessary resources and guidance that enabled the successful execution of this project.

Finally, I would like to thank my family and colleagues for their continuous encouragement and support throughout this journey.

I would like to acknowledge the assistance of OpenAI's ChatGPT in helping to improve the clarity and grammar of portions of this thesis.

Oulu, May 27, 2025

Gihan Wansekara

LIST OF ABBREVIATIONS AND SYMBOLS

AI	Artificial intelligence
AI-RAN	Artificial intelligence radio access network
AoA	Angle of arrival
AoD	Angle of departure
AR	Augmented reality
CAM	Common agreement-based method
CIR	Channel impulse response
CUDA	Compute unified device architecture
DBSCAN	Density based spatial clustering of applications with noise
DDS	Data distribution service
DT	Digital twin
EM	Electromagnetic
FOV	Field of view
GPU	Graphics processing unit
IoT	Internet of things
ITU	International telecommunication union
LiAP	LiDAR-Integrated access point
LiDAR	Light detection and ranging
LLM	Large language model
LTE	Long term evolution
MIMO	Multiple input multiple output
ML	Machine learning
NLOS	Non line of sight
OSM	OpenStreetMap
PDP	Power delay profile
PHY	Physical layer
RGB	Red-green-blue (color model)
RF	Radio frequency
RIS	Reconfigurable intelligent surface
ROS2	Robot operating system 2
RT	Ray tracing
RX	Receiver
SDF	Signed distance function
Sionna RT	Sionna ray tracer
SMS	Short message service
THz	Terahertz
TX	Transmitter
UDP	User datagram protocol
USD	Universal scene description
USRP	Universal software radio peripheral
UTD	Uniform theory of diffraction
VR	Virtual reality
Wi-Fi	Wireless fidelity
1G	First generation
2G	Second generation
3GPP	3rd generation partnership project

4G	Fourth generation
5G	Fifth generation
6G	Sixth generation
c	Speed of light
d	Distance between source and target object
E	Electric field
$E_0(\theta, \varphi)$	Electric field phasor
FSPL	Free space path loss
minPts	Minimum number of points needed to form a dense region
N	Number of detection elements distributed over the field of view
N_r	Neighbourhood radius
r	Far field distance
Δt	Time difference between emission and the reception
$\Delta\theta$	Angular resolution of the sensor
μ	Permeability
ε	Permittivity
θ	Zenith angle
φ	Azimuth angle
θ_i	Angles of incidence
θ_t	Angle of refraction
n_i	Refractive indices
$\tau_{\perp}$	Reflection coefficient for perpendicular polarization
$\tau_{\parallel}$	Reflection coefficient for parallel polarization
$L_{ke}(v)$	Knife edge diffraction loss
E_d	Diffacted field
E_i	Incident field
v	Diffraction parameter
λ	Wavelength
ϕ	Phase shift of ray tracing signals

1 INTRODUCTION

The beginning of wireless communication is 1G (First generation) in late 1970 that relied on analog voice transmission with limited mobility and security. Now it has evolved to advanced 6G networks. Then the introduction of 2G in the early 1990s was a significant milestone in wireless communication since it is the beginning of digital communication. That provided Short message service (SMS) and better quality of voice. Then in early 2000s 3G was introduced, which provided mobile internet, higher data rates and support for video calling and web browsing applications.

With the introduction of 4G in the 2010s, wireless communication was further improved by high-speed mobile broadband, low latency and seamless connectivity. Technologies such as long-term evolution (LTE) and LTE-advanced enabled a massive growth of smartphones, cloud computing and mobile applications. The ongoing deployment of 5G networks has further enhanced wireless capabilities offering ultra-low latency, high bandwidth and massive IoT connectivity. This enabled applications such as autonomous vehicles, industrial automation and augmented/virtual reality (AR/VR) technologies.

Looking ahead, 6G, expected to emerge in the 2030s, will push the boundaries of wireless technology by integrating AI-driven network management, terahertz (THz) frequencies, ultra-low energy consumption, and fully autonomous network operations [1], [2]. With sub-millisecond latency and AI-native optimizations, 6G will enable seamless digital-physical integration through digital twins and intelligent wireless environments. As networks grow more complex, the demand for efficient, reliable, and adaptive solutions has made AI, digital twin technology, and ray tracing simulations essential for next-generation wireless systems.

The rapid advancement of wireless communication has led to a rising demand for high-speed, ultra-reliable and energy-efficient networks. Next generation 6G networks are expected to support diverse applications such as smart cities, autonomous vehicles, industrial automation and the internet of things (IoT) [5]. These applications require networks that can adapt dynamically, optimize performance in real-time and make intelligent decisions.

Traditional network optimization techniques struggle to address the growing complexity of modern wireless environments. Many existing simulation frameworks rely on static models which fail to capture real-time environmental dynamics, object movements and urban variations. Moreover, testing AI-driven wireless solutions in live networks is challenging due to potential service disruptions, high computational costs and unpredictable real-world conditions.

To overcome these challenges, digital twin (DT) technology has emerged as a transformative solution for wireless network modeling and optimization. A digital twin is a virtual copy of a real-world system that helps analyze, optimize and predict performance. In 6G networks, DTs are becoming important for improving wireless connectivity, especially by supporting AI-powered advancements [3]. They create a simulation environment where network settings, resource management and AI-driven strategies can be tested before being applied in the real world.

This research presents a digital twin-driven simulation framework that integrates LiDAR sensor data, machine learning based environmental modeling, and ray tracing for wireless communication applications. Specifically, it leverages NVIDIA Isaac Sim to construct highly realistic, physics-based virtual environments that replicate real-world scenarios dynamically. A LiDAR based perception system captures real-time environmental changes such as object movements [4]. These dynamics are processed using machine learning techniques and embedded into NVIDIA Sionna, a state-of-the-art ray tracing simulation platform, to model wireless signal propagation with high accuracy.

By combining machine learning, digital twins, and ray tracing, this research provides a scalable, intelligent, and data-driven solution for wireless network design and optimization. The proposed framework enables accurate simulation, AI-powered decision making, and predictive analysis, paving the way for the next generation of autonomous, adaptive, and energy efficient wireless systems.

1.1 Existing work

The work in [4] explains with the use of LiDAR to predict human introduced blockages in 6G networks, where the line-of-sight connection is critical. It proposes a system that detects and tracks movement, predicts future trajectories using deep learning, and identifies blockages with ray-casting techniques. The approach improves network reliability by enabling proactive adjustments, with future work focusing on real-world validation and enhanced accuracy. The paper [5] demonstrates a continuously updated, radio compatible digital twin for robotic integrated sensing and communications (ISAC) in future 6G networks. It enhances a real-time DT framework with an LLM-driven speech interface for robotic control, a 3D reconstruction system using RGB camera with depth data, and a ray tracing method for real time wireless channel modeling. These innovations enable high-resolution environmental modeling, improving localization, autonomous navigation, and ISAC performance. A live demonstration showcases remote robotic operation with real-time DT updates based on natural language commands. The work in [6] demonstrates a vision for real-time digital twins in 6G and beyond, where continuously updated digital replicas of physical wireless environments are used to enhance communication and sensing decisions. Enabled by precise 3D maps, multi-modal sensing, real-time ray tracing, and machine learning, these digital twins aim to improve wireless network efficiency, reduce latency, and optimize decision-making at various network layers. The study discusses different approaches to constructing and utilizing DTs, their applications in wireless communication, and key research challenges. It also introduces a research platform for further investigation into digital twin applications in next-generation networks. The paper [7] presents Sionna ray tracer, a differentiable ray tracer for radio propagation modeling, integrated into the open-source Sionna library. It enables gradient-based optimization of system parameters like antenna configurations, material properties, and transmitter orientations, making it a powerful tool for 6G research and digital twin networks. By integrating machine learning, it supports applications such as radio-based localization, reconfigurable intelligent surfaces (RIS), and integrated sensing and communications (ISAC). The paper showcases use cases like learning radio materials and optimizing transmitter orientation to enhance wireless network performance.

1.2 Thesis contribution

The primary contribution of this research work is the design, development, and implementation of a comprehensive digital twin framework for AI-driven radio access networks (AI-RAN) in the context of 6G networks. This framework introduces a novel methodology for capturing, modelling, and simulating the complex dynamics of outdoor radio environments. It leverages machine learning algorithms to detect and interpret environmental dynamics in real time, enabling intelligent adaptation of the network. A key innovation lies in the integration of a ray-tracing simulation engine, which is used to emulate the dynamic behaviours of wireless signal propagation in realistic outdoor scenarios and to estimate channel properties with high accuracy. These estimated properties can be fed back into the network control loop, providing the AI-RAN with a rich and responsive virtual environment for continuous learning, optimization, and decision-making. By bridging physical and virtual domains through highly accurate digital twins, this research paves the way for a more robust, adaptive, and intelligent 6G infrastructure capable of responding to dynamic real-world conditions with unprecedented efficiency.

1.3 Thesis outline

This thesis is structured into six chapters and remaining chapters have been arranged as following.

- **Chapter 2:** Theoretical background of digital twins, NVIDIA tools, machine learning algorithms for dynamic detection, ray tracing techniques and electromagnetic wave properties are discussed in this chapter.
- **Chapter 3:** In this chapter, prior work related to the implementation of a digital twin such as dynamic detections, Lidar based LoS detection and different ray tracing techniques are discussed
- **Chapter 4:** Each element of the digital twin such as dynamic environment sensing, dynamic detection, emulating dynamics on a ray tracing environment and ray tracing is described here. Also, implementation of each section is discussed in this chapter.
- **Chapter 5:** This section discusses all the results obtained from the implementation presented in Chapter 4. This contains results for different frequency settings and for different dynamic environments.
- **Chapter 6:** This chapter presents the conclusion of the study along with suggested directions for future work.

2 THEORETICAL BACKGROUND

2.1 Digital twin

A digital twin is a virtual representation of a physical system that replicates the real-world conditions. DTs facilitate analysis, optimization and prediction to enhance system performance. In the context of 6G networks, DTs are becoming crucial for replicating various network layers to improve wireless system performance by facilitating AI-driven features. By using a digital environment, DTs support advanced network monitoring, simulation and optimization, where network configurations, resource management methods and AI algorithms can be tested and optimized before actual deployment.

Modern networks generate vast amounts of data from every connected device, necessitating AI and machine learning (ML) techniques for efficient and sustainable network management, starting from physical layer (PHY) signal processing. However, training and integrating AI/ML models present challenges, particularly for real-time validation and performance assurance without disrupting live networks. In these scenarios, DTs play an important role by giving a digital replica of real-world environments that are integrated with AI radio access networks to interact with AI/ML workflows. This makes DTs a crucial part of AI-RAN, facilitating networks to dynamically adapt to user demands, optimizing resource allocation and making predictive and data driven decisions. Ultimately, this enhances efficiency, reliability, and energy savings across various complex and advanced use cases in future networks.

DTs can be implemented at different levels depending on the targeted application. From a top-down perspective, a Radio access network digital twin is a virtual replica of the radio access network, including components such as base stations, antennas, and signal processing algorithms. Compared to traditional network simulators, a DT network establishes real-time communication between the physical network and its virtual twin, promising continuous interactive mapping. This enables real-time simulation and optimization of key network functions, such as resource allocation, user scheduling and load balancing.

A crucial aspect of RAN DTs is the creation of a realistic and scalable digital model of the live radio frequency (RF) propagation channel between the network and user equipment. RF DTs capture the complexities of RF interactions, such as signal propagation, attenuation, interference and multipath effects, enabling precise real-world simulations.

2.2 Light detection and ranging (LiDAR)

Light detection and ranging (LiDAR) is a remote sensing mechanism that uses light (laser) pulses to measure the distances by considering the timing delay between pulse emission and return. This method is a highly accurate 3D spatial measurement technique, that leads to the basis for point cloud generation. LiDAR is widely used in robotics, autonomous vehicles, surveying, topographic mapping, forestry and urban planning.

2.2.1 Principles of LiDAR sensing

The fundamental mechanism in LiDAR is based on the time-of-flight principle. When a laser emits a pulse toward a target object, then it reflects away from the object and returns to the LiDAR sensor as shown in the Figure 1. The time difference between pulse emission and reception is used to determine the distance between the transmitter and the target object.

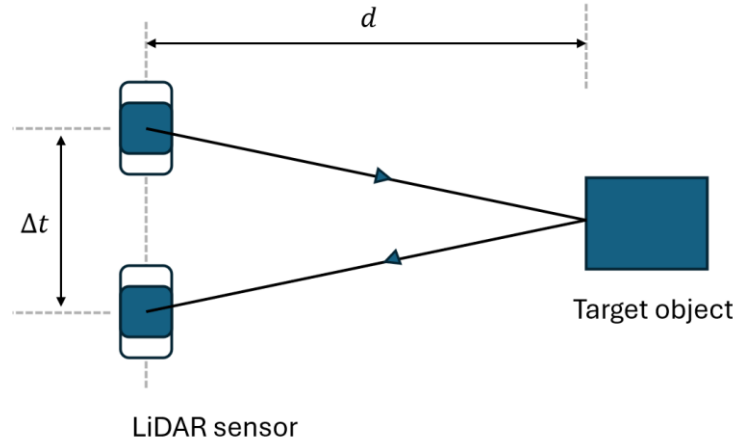


Figure 1: Time of flight principle applied to a laser pulse.

The distance between LiDAR sensor and the target object can be found using the following equation:

$$d = \frac{c\Delta t}{2} \quad (1)$$

Where c is the speed of light and Δt is the time difference between emission and the reception.

2.2.2 Angular resolution and field of view

LiDAR sensors usually rotate or use a solid-state array to cover a wide field of view (FOV). The smallest detectable angular separation is known as the angular resolution of the sensor ($\Delta\theta$). It can be determined by the equation:

$$\Delta\theta = \frac{FOV}{N} \quad (2)$$

Where N is the number of laser pulses or detection elements distributed over the field of view.

2.2.3 Intensity and reflectivity

In addition to distance, the intensity of the returned signal is recorded. This intensity is affected by the reflectivity of the target object, the angle of incidence and atmospheric conditions. This information can be used to infer material properties or enhance the segmentation of point cloud data.

2.2.4 LiDAR sensing process

LiDAR sensors emit millions of laser pulses per second. These pulses cover a specific scanning area, and the high repetition rate enables rapid data collection over large areas. Each emitted pulse undergoes the following steps.

- Emission: A laser pulse is directed toward the scene
- Reflection: The pulse strikes surfaces and is scattered back
- Detection: The sensor detects the return signal and measures the time delay and the intensity
- Conversion: Using the time-of-flight equation, the distance is calculated

This process gives a series of raw data points, each with a distance value and corresponding angular information.

2.3 Point cloud generation

A point cloud is an assembly of points in a three-dimensional space that collectively represent the surfaces of objects and environments. The generation of a point cloud involves several key processes: data integration by combining the 3D coordinates from successive pulses to form a comprehensive spatial dataset [10]; achieving high resolution and density, as high-resolution LiDAR systems produce dense point clouds that capture minute details of the environment; and rigorous preprocessing, including noise filtering, outlier removal, and normalization to ensure data accuracy [8]. Advanced segmentation and classification techniques employing clustering and machine learning algorithms then divide the point cloud into distinct objects or surfaces, a process that is especially useful for differentiating between natural and man-made structures. Finally, the processed point cloud can be visualized through methods such as 3D scatter plots, surface reconstructions, or mesh generation, which facilitates further analysis and interpretation. Point cloud generated using LiDAR data plays a crucial role in high-fidelity digital twin modelling by giving precise spatial and structural information.

2.4 NVIDIA Isaac Sim

NVIDIA Isaac Sim is a cutting-edge 3D simulation platform developed by NVIDIA, which is designed to accelerate the building, testing, validation and calibration of robotics and artificial intelligence applications. Isaac Sim is built on NVIDIA omniverse framework and that provides a highly realistic and accurate virtual environment which is the replica of the physical world environment. By using that virtual environment, robots can be modelled and tested before deployment in the real world [13]. It integrates advanced simulation capabilities such as high-fidelity physics, photorealistic rendering and realistic sensor emulation, enabling developers to test algorithms and robotic systems in complex, dynamic scenarios. By leveraging GPU acceleration and a modular, extensible architecture, Isaac Sim significantly reduces the time and cost associated with building and iterating robotic applications, thereby supporting rapid innovation in autonomous systems and intelligent machines.

Regarding wireless communication digital twins, this concept refers to the creation of virtual replicas of wireless networks or communication systems that mirror their physical counterparts in real time. With a wireless communication digital twin, engineers and researchers can simulate, monitor, and optimize network performance in a controlled virtual environment for example as shown in Figure 2. This approach enables the analysis of propagation characteristics, interference patterns, and dynamic channel conditions without the need to physically deploy or alter network infrastructure. Furthermore, digital twins facilitate predictive maintenance, capacity planning, and performance tuning by integrating data analytics, machine learning and real-time feedback. In practical applications, a wireless communication digital twin can be used to test new network configurations, experiment with emerging protocols, and evaluate the impact of environmental changes or user behaviour on connectivity. The integration of such digital twin technology with simulation platforms like NVIDIA Isaac Sim could further enhance the fidelity of system models by incorporating both physical and communication aspects, leading to more robust designs for next-generation wireless networks and smart connected systems.

NVIDIA Isaac Sim's sensor suite offers a comprehensive and highly realistic set of virtual sensors that closely mimic their physical counterparts, essential for both robotics applications and the development of wireless communication digital twins. For example, the RTX LiDAR sensor in Isaac Sim produces detailed 3D point clouds with high spatial resolution and accuracy, enabling precise mapping and obstacle detection in complex environments. In addition, high-fidelity camera systems replicate real-world imaging capabilities, including accurate colour rendering, dynamic range, and depth perception, which are vital for visual navigation and sensor fusion. Other sensors such as radar and ultrasonic devices further enrich the simulation environment by providing additional layers of environmental feedback. When these sensors are integrated into wireless communication digital twins, they allow for real-time data acquisition and analysis, which is critical for simulating network dynamics, assessing signal propagation, and optimizing channel performance under varying environmental conditions. This synthesis of advanced sensor simulation with digital twin technology ensures that both the physical and communication aspects of the system are accurately modelled, fostering more reliable and robust development of next-generation wireless and autonomous systems.

NVIDIA Isaac Sim's sensor suite is designed not only to emulate real-world sensing but also to seamlessly integrate with modern robotics software frameworks. The rich sensor data generated by virtual sensors like RTX LiDAR and high-fidelity cameras can be published as ROS2 (robot operating system 2) messages, using standardized formats. This integration enables real-time data exchange between simulation and processing nodes, allowing developers to leverage

ROS2's powerful communication infrastructure for tasks such as sensor fusion, perception, and decision-making. By publishing sensor data as ROS2 messages, Isaac Sim facilitates a smooth transition from virtual testing to real-world deployment, enhancing the development of robust autonomous systems and wireless communication digital twins.



Figure 2: Simulation of human walking in Issac Sim.

2.5 ROS2 for data communication in digital twin frameworks

ROS2, the next-generation Robot operating system, is a flexible, open-source middleware framework designed to enable robust and real-time communication in distributed systems, making it particularly well-suited for digital twin frameworks. By leveraging a publisher-subscriber model alongside service and action paradigms, ROS2 facilitates seamless, asynchronous data exchange across diverse devices and applications. Its integration of the data distribution service (DDS) standard enhances network performance, reliability, and security, which is essential for maintaining synchronized, bidirectional communication between physical systems and their digital counterparts. Moreover, the architecture of ROS2 supports advanced quality-of-service settings that allow developers to prioritize and manage data flows effectively, ensuring that critical information is transmitted without delay or loss. This makes ROS2 an ideal solution for complex digital twin applications where accurate, real-time data communication is paramount for simulation, monitoring, and control tasks.

2.6 Clustering and dynamic object detection using DBSCAN

DBSCAN (Density-based spatial clustering of applications with noise) is a powerful clustering algorithm that excels in identifying arbitrarily shaped clusters and filtering out noise, making it

highly suitable for dynamic object detection in various environments [11]. In dynamic object detection, DBSCAN is applied to spatial data such as 3D point clouds from LiDAR sensors or pixel data from video streams to group data points that are closely packed together into clusters, which can represent moving objects like vehicles, pedestrians, or animals [12]. The algorithm uses two key parameters: the neighbourhood radius (Nr) and the minimum number of points ($minPts$) needed to form a dense region. By iteratively expanding clusters from high-density areas and distinguishing between core points, border points, and outliers, DBSCAN effectively isolates dynamic objects from background noise and static elements. Additionally, LiDAR scanned data is frequently integrated within the ROS2 environment, which offers a robust framework for processing sensor data and developing autonomous systems. In this context, DBSCAN can also be implemented in ROS2, enabling real-time clustering and dynamic object detection for applications in autonomous driving, robotics, and surveillance, where the processing of high-dimensional sensor data in a complex, ever-changing environment is critical.

2.6.1 DBSCAN algorithm

In DBSCAN algorithm, neighbourhood radius (Nr) and the minimum number of points ($minPts$) are the two main controlling parameters. There are three types of points considered in this algorithm.

1. Core points
2. Border points
3. Noise points

As shown in Figure 3, Core point is a point that has enough neighbouring points lower bounded by $minPts$ within the radius defined by Nr . Border points are the points placed near the core points but do not have enough neighbouring points to be considered as core points. By considering core points and border points, clusters can be formed. Noise points are the points that do not belong to any cluster.

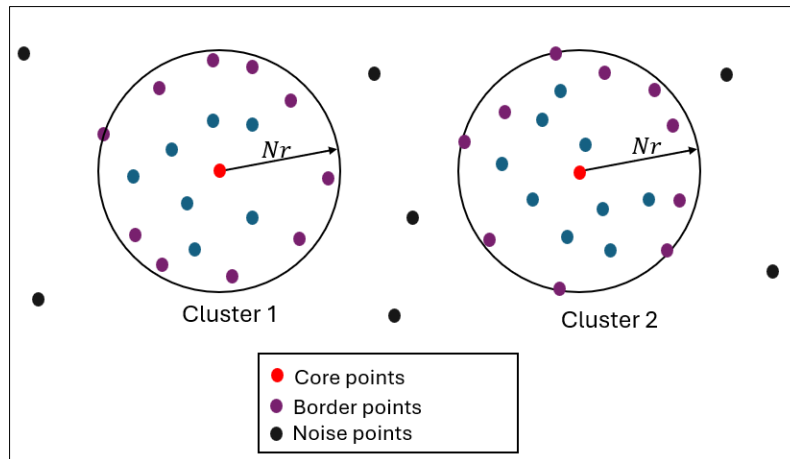


Figure 3: DBSCAN clustering

The following steps describe DBSCAN algorithm workflow.

1. Initialization and neighbourhood identification

The algorithm starts by iterating over every point in the dataset. For each point, it finds all neighbouring points that lie within the distance Nr .

2. Identification of core points

A point is considered a core point if the number of points in its Nr neighbourhood is at least $minPts$

3. Formation of clusters and expansion

When a core point is found, a new cluster is started, and the algorithm expands the cluster by considering all its neighbours. For each neighbour if it has not been visited, it is marked as visited and its own Nr neighbourhood is examined. If that neighbour also qualifies as a core point, then its neighbours are added to the cluster expansion process. This cluster expansion process ends when it reaches the boundary points.

4. Identification of noise

If a point is not a core point and is not density-reachable from any other core point within the Nr neighbourhood, then it is marked as noise.

2.7 Fundamentals of ray tracing

Ray tracing in wireless communications is a deterministic modelling technique that simulates the propagation of electromagnetic waves through an environment by treating them as rays. This method is particularly useful for complex environments (urban, indoor, or mixed scenarios) where waves undergo multiple reflections, diffractions, and refractions.

In ray tracing, electromagnetic waves are modelled as rays that travel in straight lines between interactions (such as reflection, refraction, or diffraction) [14]. Each ray carries amplitude and phase information. Also, the method relies on the high-frequency approximation of the wave equation, where the wavelength is much smaller than the dimensions of the obstacles. This allows the users to neglect wavefront curvature over small distances and treat propagation using geometric optics principles. The beginning of the ray tracing is the homogeneous wave equation for the electric field E

$$\nabla^2 E - \mu\epsilon \frac{\partial^2 E}{\partial t^2} = 0 \quad (3)$$

Where μ is the permeability and ϵ is the permittivity. In the context of ray tracing, this equation is simplified by assuming that the fields propagate along rays with a well-defined direction, and the phase of the wave evolves according to the distance travelled.

2.7.1 Far field assumption of a transmit antenna.

Electric field of an antenna at far field can be expressed by a spherical wave which was originated from the center of the antenna.

$$E(r, \theta, \varphi, t) = E_0(\theta, \varphi) \frac{e^{-jk_0 r}}{r} e^{j\omega t} \quad (4)$$

Where r is the far field distance, $E_0(\theta, \varphi)$ is electric field phasor and θ, φ are zenith and azimuth angles respectively.

2.7.2 Reflection and refraction

When a ray encounters an interface between two media with different electromagnetic properties, following two phenomena occur as given in Figure 4:

- **Reflection:** A portion of the wave is returned into the medium which it originated.
- **Refraction (transmission):** The remainder of the wave penetrates the second medium, with its direction altered according to Snell's law.

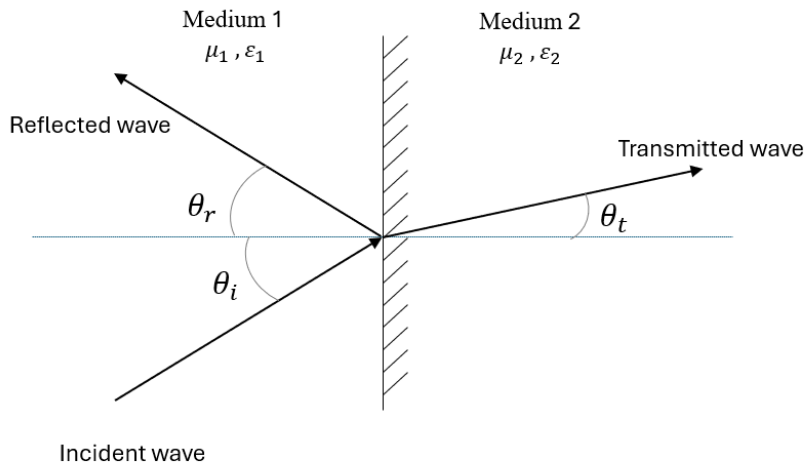


Figure 4: Incident of a plane wave onto a plane boundary

Snell's law relates the angles of incidence θ_i and refraction θ_t to the refractive indices n_1 and n_2 of the two media with the following equation.

$$n_1 \sin \theta_i = n_2 \sin \theta_t \quad (5)$$

2.7.3 Fresnel coefficients

The amplitude of the reflected and transmitted waves is determined by the Fresnel reflection and transmission coefficients. For a plane wave incident on a flat boundary, these coefficients depend on the polarization of the incident wave.

- Perpendicular (s-polarized) waves:

$$\tau_{\perp} = \frac{n_1 \cos \theta_i - n_2 \cos \theta_t}{n_1 \cos \theta_i + n_2 \cos \theta_t} \quad (6)$$

- Parallel (p-polarized) waves:

$$\tau_{\parallel} = \frac{n_2 \cos \theta_i - n_1 \cos \theta_t}{n_2 \cos \theta_i + n_1 \cos \theta_t} \quad (7)$$

These coefficients indicate how much of the wave's amplitude is reflected at the interface. The transmitted (or refracted) wave's amplitude is similarly computed from transmission coefficients which are complementary to the reflection coefficients.

2.7.4 Diffraction

Diffraction is the bending of waves around obstacles or apertures. In ray tracing, especially for environments with sharp edges (like buildings or walls), the knife-edge diffraction model is often used. The knife edge diffraction loss ($L_{ke}(v)$) can be expressed in decibels, in terms of a diffraction parameter, v given in the following equation.

$$L_{ke}(v) = -20 \log \left(\left| \frac{E_d}{E_i} \right| \right) \quad (8)$$

where E_d is the diffracted field and E_i is the incident field and v depends on the geometry of the obstacle and the wavelength.

2.7.5 Scattering

Electromagnetic scattering is the redirection of electromagnetic waves caused by objects or irregularities in the propagation medium. When an incident wave encounters a particle or surface especially one whose dimensions are comparable to or smaller than the wavelength then it induces secondary waves that radiate in various directions (Figure 5). The nature of scattering depends on the size, shape, and material of the object, as well as the frequency of the wave. It can be categorized into Rayleigh, Mie, or geometric scattering, depending on the size-to-wavelength ratio. In wireless communication, scattering contributes to multipath propagation, where the received signal is a superposition of multiple scattered waves, affecting signal strength and

quality. While scattering can lead to signal fading, it also enables non-line-of-sight (NLOS) transmission, making it a crucial phenomenon to model in realistic environments.

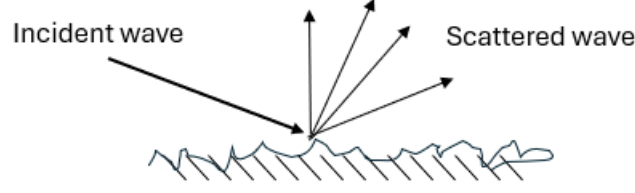


Figure 5: Rough surface scattering

2.7.6 Path loss and phase calculation

For a ray that travels in free space (with no interactions), the free space path loss (*FSPL*) quantifies the reduction in power density with distance d and wavelength λ :

$$FSPL = \left(\frac{4\pi d}{\lambda} \right)^2 \quad (9)$$

Each ray accumulates a phase shift as it propagates. The phase shift ϕ is given by:

$$\phi = \left(\frac{2\pi d}{\lambda} \right) \quad (10)$$

This phase factor is crucial when summing multiple ray contributions at the receiver, as constructive or destructive interference will occur depending on the phase differences.

2.7.7 Ray launching and propagation

In ray launching, rays are typically emitted from the transmitter in multiple directions. The density of rays can be adapted based on the desired resolution of the simulation. As rays encounter surfaces, the algorithm computes the new direction, amplitude, and phase based on reflection, refraction, or diffraction models. In complex environments, rays may undergo several interactions. The cumulative effect is determined by sequentially applying the above equations at each interaction point. At the receiver, the contributions from all arriving rays are superposed. This summation considers both the amplitude attenuation (from path loss and interactions) and the phase shifts:

$$E_{total} = \sum_{i=1}^N E_i e^{-j\phi_i} \quad (11)$$

Where, E_i and ϕ_i are the amplitude and phase of the i^{th} ray respectively.

Ray tracing in wireless communication simulates electromagnetic wave propagation using geometric optics and key electromagnetic equations. This method enables engineers to predict signal behaviour, address multipath interference, and optimize network performance through both visual and quantitative analyses.

2.8 NVIDIA Sionna ray tracing

NVIDIA Sionna Ray Tracing (Sionna RT) is a high-performance simulation tool that models how electromagnetic waves travel through complex environments. It uses ray tracing, a technique from computer graphics, to track how radio waves bounce, bend, and scatter when they hit objects like buildings, walls, or furniture as shown in Figure 6. Unlike simple wireless models that estimate signal behavior with averages, Sionna RT uses detailed 3D maps and real material properties to calculate each possible signal path between a transmitter and a receiver. This makes it very accurate for simulating real-world wireless channels.

Sionna RT is built on top of NVIDIA's OptiX engine and runs on GPUs, which allows it to handle large environments and simulate thousands of rays very quickly. It supports important wireless effects such as reflection, diffraction, and scattering, and computes detailed results like the path loss, angle of arrival (AoA), angle of departure (AoD), delay spread, and power delay profiles (PDP). It can model frequency dependent behaviour and works well with advanced antenna systems like MIMO and beamforming. This helps engineers design and test technologies used in 5G, 6G, and Wi-Fi systems.

One of the key strengths of Sionna RT is that it connects well with machine learning workflows. It can generate realistic training data for AI models used in wireless communication tasks like channel estimation, user localization, and intelligent beam selection. Because of its accurate physics and fast GPU processing, Sionna RT is especially useful for simulating modern scenarios such as indoor offices, urban streets, and even systems using reconfigurable intelligent surfaces (RIS). It's a powerful tool for researchers and engineers building the next generation of wireless networks.

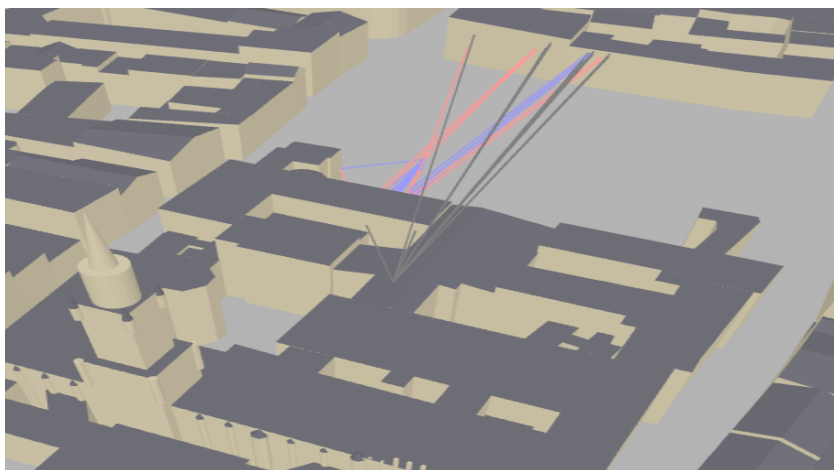


Figure 6: Ray tracing on Sionna

3 RELATED WORK

This chapter will review the previous research work done which will lead to the development of a dynamic digital twin for wireless communication. Here different phases of the digital twin such as dynamic detection, ray tracing techniques will be reviewed.

3.1 LiDAR aided human blockage prediction for 6G

This study [4] addresses the key challenge of line-of-sight interruptions at millimeter-wave and terahertz frequencies, which are fundamental to 6G systems. Due to the shorter wavelengths at these high frequencies, communication links are highly sensitive to dynamic blockages particularly those caused by human movement. To tackle this, the authors propose an innovative infrastructure-based system using elevated LiDAR sensors to capture detailed 3D data of the environment. Their system processes LiDAR point clouds to detect, track, and forecast human motion using deep learning models, and evaluates future LOS conditions via a lightweight ray casting technique.

The core of the system comprises three stages such as real-time human detection and tracking through point cloud clustering and Kalman filtering, motion trajectory prediction using long short-term memory (LSTM) networks to model sequential human movement patterns, and ray-box intersection analysis to detect potential signal blockages by projecting rays from transmitters toward predicted human locations. Importantly, the method operates entirely on visual data, minimizing reliance on wireless-side feedback. The system demonstrates strong performance in simulation, achieving 87% accuracy, 78% precision, and 79% recall for blockage prediction over a 300 ms window indicating its practical value in enabling proactive beam switching or handovers in dense 6G indoor deployments.

This work builds upon and extends prior studies involving LiDAR and camera-based sensing for beam selection and channel estimation in mmWave environments. Unlike earlier approaches that focused on reactive adaptations, the proposed method emphasizes proactive prediction using high-resolution 3D sensing, demonstrating a meaningful step forward in integrating vision-based intelligence into future wireless systems. The use of object-oriented bounding boxes (OBB) and real-time trajectory forecasting adds robustness to the blockage prediction, while also revealing promising opportunities for integrating vision sensors with network-side control logic in next-generation communication systems.

3.2 LoS detection and prediction based on static maps

In this work [16], it is proposed that the integration of LiDAR sensors directly into access points (LiAPs) to enable vision-based LoS detection and proactive link management. Also it is highlighted that at higher frequencies such as those used in 5G and emerging 6G systems signals experience severe attenuation and are highly sensitive to blockages. These impairments need a sensing mechanism that can monitor the wireless environment in real-time and inform the network of potential disruptions. By leveraging LiDAR's precise 3D point cloud capabilities, their system can detect users, determine LoS status, and predict future LoS transitions before they occur.

The proposed solution consists of two major contributions such as a common agreement-based method (CAM) for multi-sensor fusion to improve user detection accuracy and a static-

map-based LoS prediction model that identifies potential LoS-to-NLoS transitions caused by static obstacles. CAM enhances the robustness of user detection by aggregating reports from multiple LiAPs, using a voting mechanism to confirm detections and filter out false positives. Once user positions are reliably identified, the system extracts transition boundaries regions in the static environment where LoS conditions are likely to change. These boundaries are derived by analyzing discontinuities in 2D projections of LiDAR point clouds and refined using techniques like RANSAC line fitting. The system then monitors user movement relative to these boundaries to estimate the likelihood of an impending LoS transition.

By computing metrics such as the shortest distance to a transition boundary and its rate of change, the authors developed a lightweight predictor that anticipates LoS degradation up to 400 ms in advance. This prediction window allows the network to proactively reconfigure beam paths or perform handovers before a link failure occurs. Simulation results in realistic 3GPP-inspired office layouts demonstrate that the CAM method reduced user misdetection probability from 31% to 14% and improved localization accuracy by 5 cm. These outcomes showcase the effectiveness of embedding sensing capabilities within network infrastructure and establish a foundation for smarter, more resilient 6G systems.

3.3 Ray tracing based radio channel modelling applied to RIS

This paper [15] presents a ray tracing-based simulator tailored for deterministic radio channel modeling, with a specific focus on evaluating the performance of reconfigurable intelligent surfaces in urban environments. Their work addresses the growing need for physically accurate and computationally efficient simulation tools in 5G and emerging 6G wireless networks operating in the mmWave and sub-THz bands. Unlike traditional stochastic models, the authors adopt a geometry-based approach using NVIDIA's GPU-accelerated OptiX ray tracing engine and CUDA for high-performance computation. The core contribution lies in their two-stage ray tracing method, which first performs a coarse visibility analysis through discretized environment mapping and then refines selected paths to compute exact electromagnetic interactions, including reflections and diffractions.

The RIS is modeled as a uniform planar array of phase-controllable reflecting elements. Each element introduces a specific phase shift to the incoming signal, enabling dynamic beamforming toward the receiver. The authors compute three primary types of path combinations TX-to-RIS, RIS-to-RX, and direct TX-to-RX allowing the simulator to evaluate signal propagation under varying visibility conditions. The simulator computes the output electric field by chaining polarimetric transformation matrices and applying wavefront spreading factors, following the principles of geometric optics and the uniform theory of diffraction (UTD). The RIS model is further refined using a phased-array formulation, assuming far-field conditions for simplified computation while retaining physical realism.

Simulation experiments in a realistic 3D model of Oulu, Finland, demonstrate that a well-positioned RIS can significantly enhance received signal power by several tens of decibels compared to non-RIS scenarios, provided that both the transmitter and receiver maintain line-of-sight (LoS) to the RIS. Even in partially blocked or non-line-of-sight (NLoS) cases, coverage gains were observed, although less pronounced. The authors also highlight the sensitivity of RIS performance to beam misalignment, reinforcing the importance of precise steering algorithms. Their results validate the effectiveness of ray tracing for modeling RIS-assisted links and underscore the potential of RIS technology in improving spectral efficiency and coverage in dense urban deployments.

3.4 A ray launching approach for computing exact paths with point clouds

In this work [17], unlike traditional ray tracing techniques that rely on triangle-mesh models, this approach directly processes labeled point clouds, which are more reflective of real-world sensing data captured from LiDARs and depth cameras. This capability is crucial for 6G wireless communication and integrated sensing applications, where accurate and timely environmental modeling is essential. The proposed method stands out for its GPU-accelerated architecture and reduced pre-processing time, making it particularly suitable for dynamic and cluttered environments like indoor corridors or urban streetscapes.

The algorithm consists of three core components: voxelization, coarse path tracking, and path refinement. The voxelization phase discretizes the 3D point cloud into multi-resolution voxel grids, enabling efficient spatial traversal and intersection testing. The coarse path tracking module casts rays from transmitters across the environment to estimate paths involving reflections and diffractions using voxel cone tracing. These coarse paths are then refined into physically accurate trajectories using optimization techniques that minimize total path length based on interaction points. This process also includes the computation of signed distance functions (SDFs) for surface approximation and Fresnel zone filtering for eliminating redundant paths. The system supports both line-of-sight (LoS) and non-line-of-sight (NLoS) propagation modeling with diffraction edge handling, a significant advancement over previous environment-driven methods.

Experimental validation was performed by comparing the output of the proposed simulator against a commercial ray tracing tool in synthetic environments. The results showed a high match rate for paths, with 100% correspondence for reflected paths and over 85% for diffracted paths. The refinement phase significantly improved the physical realism of paths, while the algorithm-maintained execution efficiency even in high-resolution settings. This work demonstrates the feasibility of high-accuracy radio channel modeling directly from point clouds, bridging the gap between vision-based sensing and deterministic wireless simulation. It lays a strong foundation for real-time digital twin implementations and context-aware communications in future 6G networks.

3.5 Differentiable ray tracing for radio propagation modeling

This paper [7] introduces Sionna RT, the first differentiable ray tracing engine designed for realistic and gradient-aware radio propagation modeling. Integrated within the open-source Sionna library and built on top of TensorFlow and Mitsuba 3, Sionna RT enables not only conventional ray tracing but also the computation of gradients of channel characteristics such as channel impulse responses (CIRs), coverage maps, and propagation paths with respect to physical and system parameters. This makes it a powerful tool for end-to-end system optimization and machine learning applications in 6G networks, such as transmitter orientation tuning, antenna configuration, and digital twin creation.

Unlike traditional stochastic models, which fail to provide spatial consistency and physical interpretability, Sionna RT simulates the electromagnetic wave behavior using high-resolution 3D scenes and customizable radio material properties (e.g., permittivity and conductivity). It supports key propagation effects, including specular and diffuse reflection, as well as first-order diffraction. One of its major innovations is its full differentiability, enabling gradient-based optimization of environment-aware parameters. For example, it allows training material properties or antenna orientations by minimizing the difference between observed and simulated

CIRs. The system leverages TensorFlow's automatic differentiation engine, and its outputs are seamlessly used in link-level simulations.

Practical use cases demonstrate the value of this differentiable simulation. In one application, the authors optimize unknown material properties by learning from synthetic CIR datasets, effectively "calibrating" the scene through backpropagation. In another, they apply gradient ascent to optimize the orientation of a transmitter for maximizing average received power in a designated region. These examples highlight how Sionna RT acts as a bridge between physics-based modeling and modern machine learning techniques, enabling novel approaches in wireless design such as inverse rendering, radio scene reconstruction, and real-time environment adaptation. As such, this work significantly advances the role of ray tracing in 6G research by making it not only accurate but also trainable and adaptable.

4 IMPLEMENTATION

In 6G, digital twins help to enhance network performance via advanced simulation techniques, optimization and the integration of AI. This chapter provides the in-detail explanation regarding the implementation of the real time digital twins of cities and industries for wireless communication specially for AI-RAN. This explains the workflow of digital twins, modelling of the physical world scenarios in Isaac sim simulation platform with sensor simulation, transferring the sensor data via ROS2 messages(topics), usage of machine learning approach to identify dynamics from the ROS2 sensor data and embedding the detected dynamics on a ray tracing environment and usage of ray tracing data for wireless communication applications.

4.1 Workflow for real time dynamic digital twins

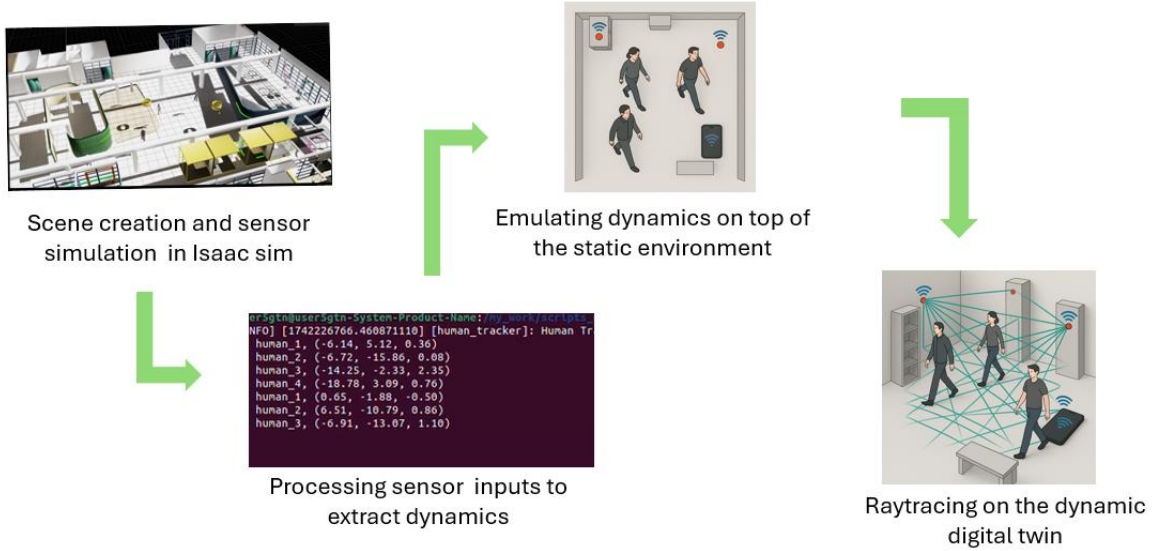


Figure 7: Workflow of a real time dynamic digital twin

As given in Figure 7, implementation of a real time dynamic digital twin can be divided into four main sub areas such as Scene creation and sensor simulation, processing of the sensor inputs to extract dynamics of the environment, emulation of dynamics on top of the static environment to create a dynamic digital twin and finally ray tracing on a dynamic digital twin so that the impact of the dynamics in the environment can be analysed and accordingly giving feedback to the base stations and other stake holders.

At first, there should be a dynamic environment for which the digital twin should be implemented. By using NVIDIA Isaac Sim, dynamic scenarios can be simulated on top of an accurate static 3D model of the desired environment. Then sensors such as LiDAR can be placed and simulated within the same dynamic digital twin to sense the environment so that dynamics can be extracted. This phase also can be replaced by real sensor inputs in a dynamic environment.

Then at the next phase, sensor inputs either from a simulation environment or from real sensors in a dynamic environment can be processed so that dynamic objects can be detected.

For this, machine learning based algorithms can be used to correctly distinguish dynamic objects from the static background.

Then at the 3rd phase, identified dynamic objects can be emulated on a ray tracing model. Usually, ray tracing utilities do ray tracing on static environments. Since the dynamics are emulated on the static ray tracing environment, ray tracing can be done on a dynamic environment as well. NVIDIA Sionna ray tracing utility supports this feature.

Finally ray tracing can be done on the dynamic environment and based on the ray tracing information, the effect of dynamics can be analysed. This data can be sent to required sections so that decisions can be taken to optimise the network.

4.2 Scene creation and sensor simulation on pre-developed 3D models in Isaac sim

It was required to first create a static digital twin of a real-world environment using NVIDIA Isaac Sim and then modelling and simulating dynamics scenarios on top of that. Initially already developed accurate 3D model of Tellus area at university of Oulu was found [18] and that was used in the initial stage to check the feasibility of using already available 3D models for static digital twins which is given in Figure 8. NVIDIA Isaac Sim supports universal scene description (USD) format, and most of 3D models including Tellus model are available in USD format.

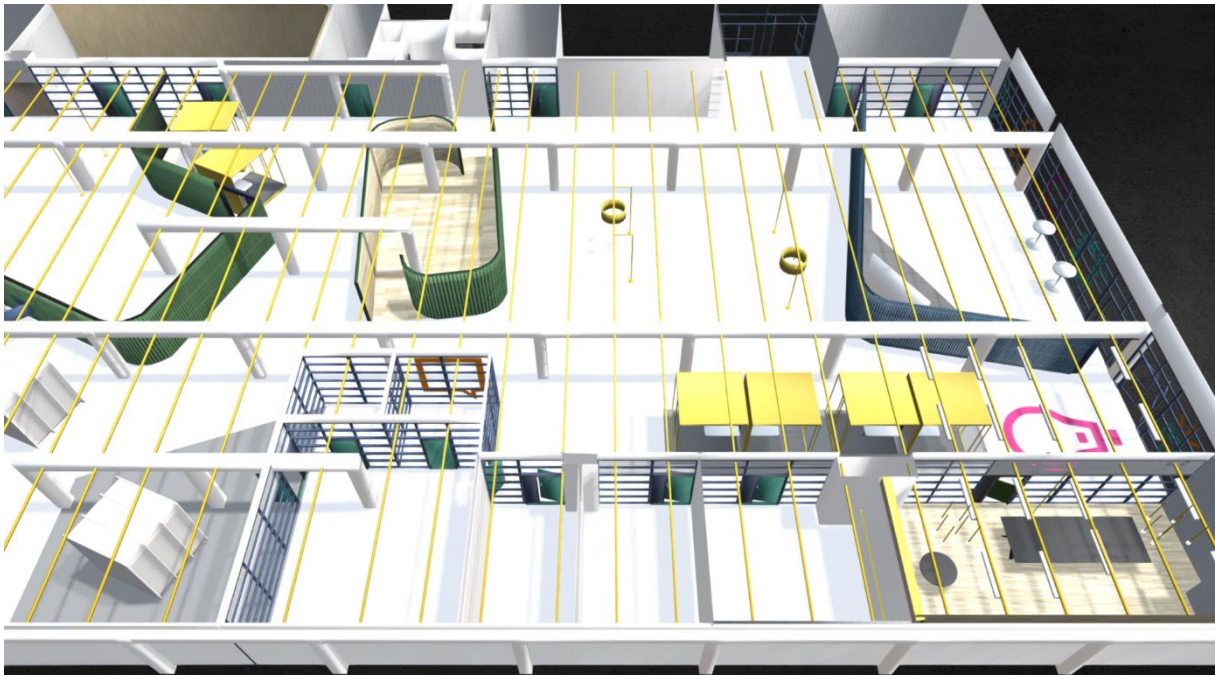


Figure 8 : 3D model of the Tellus area imported in Isaac Sim

4.2.1 Human dynamic simulation in Isaac Sim

In Isaac Sim, there are multiple extensions that support the simulation of dynamic events. ‘Omni.Anim.People’ extension is one of such that facilitates the human modelling and motion simulation such as walking, looking around, standing, etc for different environments. With the aid of this extension, the user can define multiple characters and their positions with [X, Y, Z] coordinate systems as in figure 9. Then user can define the actions of each character that should be done during the simulation. Then these human models will replicate the defined human behaviours during the simulation. Also, this extension supports to avoid dynamic object collision. By using this extension, human walking dynamics were created on top of the Tellus static 3D model to create a dynamic digital twin. This will be replicating the real-world dynamic environment.



Figure 9: Human dynamics creation in Isaac Sim

4.2.2 Sensor modelling in Isaac Sim

Once the dynamic simulation model is created, which replicates the real-world environment, the next task was to place the sensors within the simulation environment and to capture the environment to detect the dynamic objects. In this case, although main sensor used is LiDAR, Isaac Sim supports multiple perception-based sensors which includes LiDAR, radars and cameras that can sense the surrounding. OUSTER OS0-128 Rev 7 lidar sensor model in Isaac Sim was placed in the dynamic scene to capture the dynamics of the environment.

The Ouster OS0-128 Rev 7 (figure 10) is a high-resolution LiDAR sensor designed for applications in robotics, warehouse automation, and industrial environments. It features 128 channels with a vertical field of view of 90° (+45° to -45°) and a full 360° horizontal FOV. The sensor delivers up to 5.2 million points per second, with configurable horizontal resolution (512, 1024, or 2048 points per rotation) and rotation rates of 10 or 20 Hz. It offers a maximum range of up to 100 meters, even reaching 35 meters on 10% reflectivity targets in high sunlight

conditions, with a minimum range of 0.5 meters. The OS0-128 Rev 7 provides two returns (strongest and second strongest) and exceptional range precision (± 0.8 cm to ± 4 cm), accuracy (± 2.5 cm for Lambertian targets), and resolution (0.1 cm). It operates in extreme temperatures ranging from -40°C to $+60^{\circ}\text{C}$, has IP68 and IP69K ingress protection, and weighs approximately 500 grams. The power consumption ranges between 14–20 W. The sensor uses an eye-safe 865 nm laser with a beam divergence of 0.35° . It connects via UDP over gigabit ethernet, supporting data rates up to 254 Mbps and offers low data latency (<10 ms) and high timestamp resolution (<1 μs). Output data includes range, signal, reflectivity, near-infrared, channel, azimuth angle, and timestamps. Included with the sensor are a modular cap, sensor baseplate, interface cables, a 24V power adapter, and an Ethernet cable. This LiDAR unit is a compact, robust, and powerful solution for real-time 3D perception in demanding environments.



Figure 10: Ouster OS0-128 Rev 7 Lidar sensor [19]

The LiDAR sensor data captured can be published as a ROS2 topic. Sensor data processing unit can subscribe to these ROS2 topics and use them for dynamic detection. To do that, Isaac Sim provides action graph feature to support sensor data controls. Action Graph is a robust, node-based visual programming interface that allows users to create and manage complex robotic behaviours and data flows within simulation environments. It provides a modular framework where each node represents a distinct operation such as data processing, motion control, or communication and these nodes can be connected to define the logic and sequence of actions within a simulation.

One of the key applications of the action graph is in publishing LiDAR sensor data as point clouds to ROS 2 topics. By using built-in nodes for ROS 2 communication and sensor integration, users can simulate LiDAR input in real time and convert it into structured point cloud data. This data can then be published over ROS 2, allowing seamless interaction with external robotic systems, middleware, and perception pipelines. The action graph simplifies the process of simulating realistic sensor outputs and enables end-to-end testing of robotic algorithms in a controlled, virtual environment.

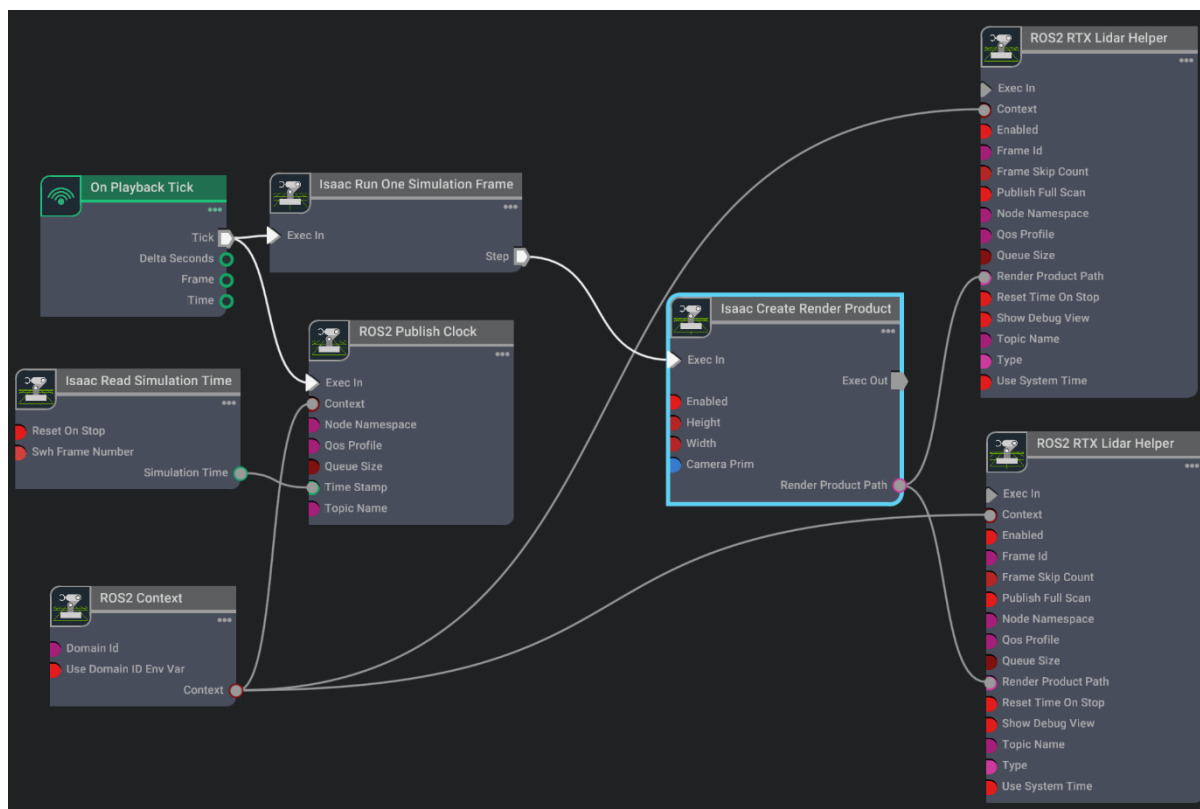


Figure 11: Action graph created for publishing LiDAR sensor data

As per the Figure 11, an action graph was created to publish LiDAR sensor data as ROS2 topics. The action graph consists of several nodes such as ‘On Playback Tick, Isaac Read Simulation Time, Isaac Run One Simulation Frame, ROS2 Publish Clock, ROS2 Context, Isaac Create Render Product’ and ‘ROS2 Lidar Helper’.

The ‘On Playback Tick’ node is a utility node that emits a signal on every simulation playback tick, effectively allowing other connected nodes to execute actions in sync with the simulation timeline. It is particularly useful for triggering behaviours or data processing steps at each frame of the simulation loop. The node outputs a tick signal that can be connected to downstream nodes to initiate operations such as publishing sensor data, updating robot states, or executing control logic continuously during playback. It provides a mechanism to synchronize graph activity with the simulation clock, ensuring time-sensitive actions are aligned with the simulation's progression.

The ‘Isaac Read Simulation Time’ node is designed to output the current simulation time, making it a vital component for time-dependent operations within a robotics simulation workflow. This node reads the time from the simulator’s clock and provides it as an output in seconds, enabling other nodes in the graph to access precise temporal information. It is particularly useful for tasks such as timestamping sensor data, synchronizing with ROS 2 messages, or implementing time-based control logic. This node makes sure that publishing messages or triggering events are accurately aligned with the simulation timeline, allowing for consistent and reproducible behaviour in both testing and deployment scenarios.

The ‘Isaac Run One Simulation Frame’ node allows controlled execution of a single simulation frame per trigger, making it ideal for precise, step-by-step data generation. When used in

workflows that publish LiDAR data to ROS 2 topics, this node ensures that each frame of the simulation produces one corresponding point cloud message, enabling accurate timing and deterministic behaviour. This is particularly useful for debugging, synchronized data output, and testing ROS 2-based perception pipelines that rely on consistent sensor data streams.

The ‘ROS2 Publish Clock’ node is used to publish the simulation time to the ‘/clock’ topic, which is essential when running ROS 2 in simulation mode. This node ensures that all ROS 2 nodes within the system are synchronized with the simulation’s internal clock, allowing time-stamped messages such as LiDAR point clouds to be accurately aligned with the simulated timeline. It is particularly important for time-dependent operations like sensor fusion, SLAM, and navigation in ROS 2-based applications.

The ‘ROS2 Context’ node initializes the ROS 2 environment within the simulation, enabling communication between Isaac Sim and external ROS 2 nodes. It sets up the necessary context for other ROS 2-related nodes such as publishers and subscribers to function correctly. This node must be present in the action graph for any ROS 2 interaction to occur, including publishing LiDAR point clouds or receiving control commands. It acts as the foundation for establishing a reliable bridge between the simulation and the ROS 2 ecosystem.

The ‘Isaac Create Render Product’ node is used to define a render product for sensors, including LiDAR, by linking the sensor to a specific viewport or render context in the simulation. When used with simulated LiDAR, this node enables the generation of point cloud data from a defined perspective, allowing the LiDAR sensor to accurately perceive the virtual environment. It is essential for producing sensor outputs that can be published to ROS 2 topics, ensuring that the data is correctly tied to the simulation’s rendering pipeline and aligned with the simulation clock for precise, time-synchronized outputs.

The ‘ROS2 Lidar Helper’ node in the action graph is specifically designed to streamline the process of publishing simulated LiDAR data to ROS 2 topics. It takes the output from a LiDAR sensor and formats it into the standard ROS 2 ‘sensor_msgs/PointCloud2’ message type, handling necessary transformations and frame settings. This node simplifies integration with ROS 2-based systems by managing key parameters like topic name, frame ID, and data rate, ensuring that point cloud data is published accurately and in sync with the simulation. It is essential for bridging simulated LiDAR sensors with ROS 2 perception or mapping pipelines. This allows users to publish the LiDAR data either as a 3-D point cloud (Figure 12) or else as a 2-D laser scan (Figure 13).

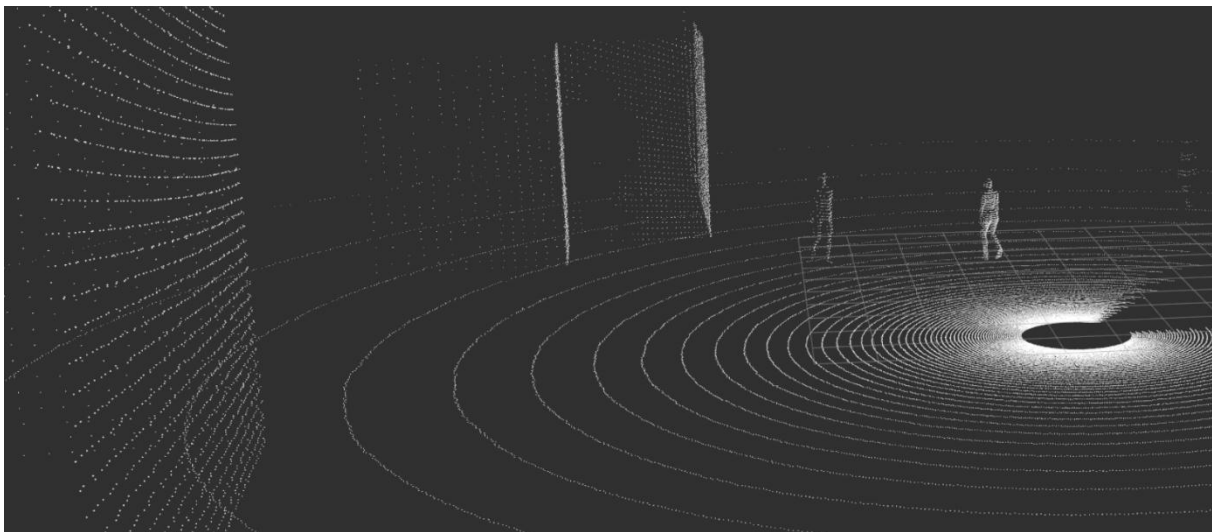


Figure 12: LiDAR point cloud output of ROS2 topic

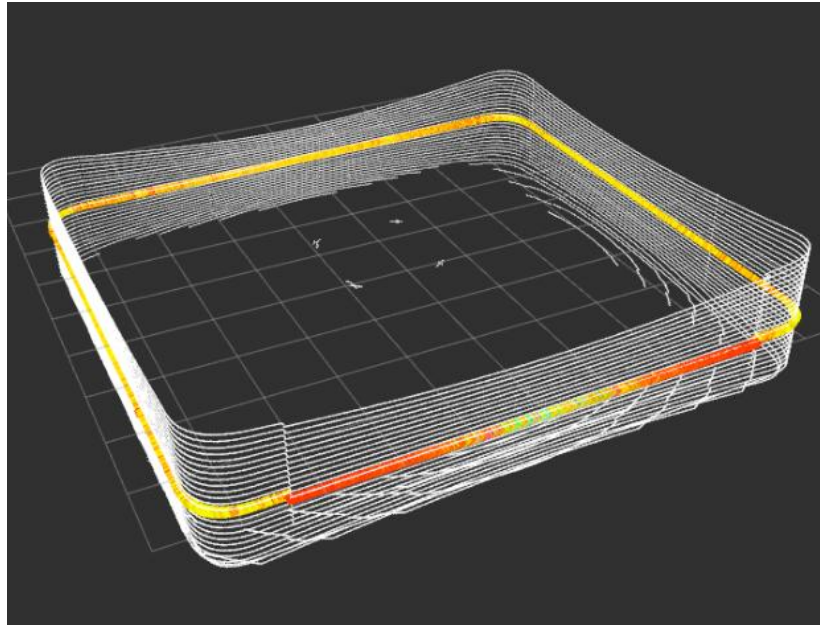


Figure 13: LiDAR laser scan output of ROS2 topic [21]

4.3 Scene creation and sensor simulation for outdoor environments

For outdoor environments, instead of having a pre-developed 3-D model, open-source geographical maps can be used with the aid of 3-D modelling Blender software. OpenStreetMap (OSM) is a collaborative, open-source mapping project that provides free and editable geographic data of the entire world. It is created and maintained by a global community of contributors who collect and upload data about roads, buildings, natural features, and other geographic elements using GPS devices, aerial imagery, or local knowledge as given in Figure 14. OSM data is widely used in applications such as navigation, urban planning, disaster response, and robotics due to its openness and flexibility. Unlike commercial mapping services, OSM allows anyone to use, modify, and share the map data freely under the Open Database License. Developers and organizations can integrate OSM into their software, customize it, and even build detailed simulation environments using its data.

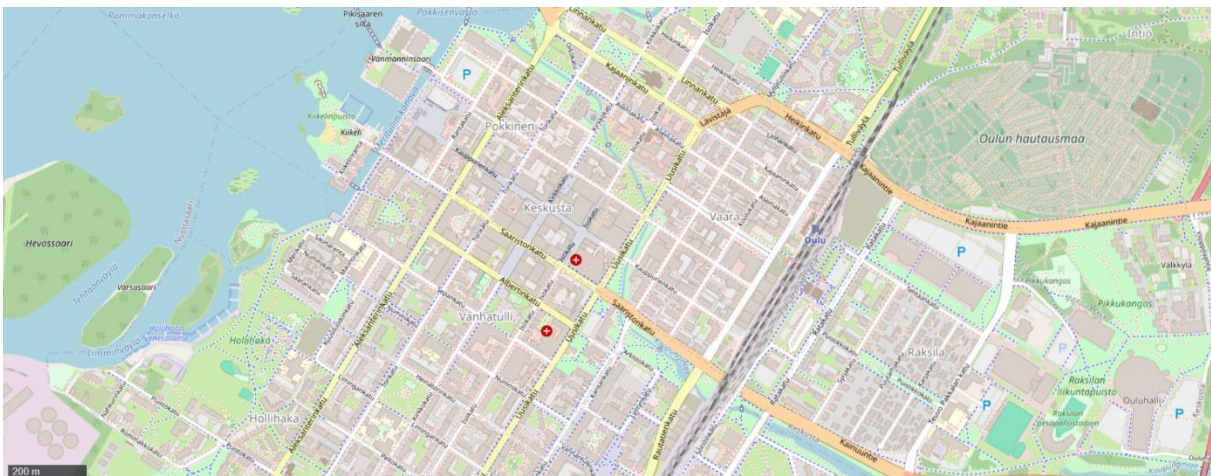


Figure 14: Oulu City view in Open Street Map [20]

Blender is a powerful, free, and open-source 3D creation software used for modelling, animation, rendering, visual effects, game development, and more. It offers a comprehensive suite of tools that allow users to create 3D models.

To generate outdoor environment within Blender, OSM data should be downloaded to Blender from platforms such as openstreetmap.org which provide detailed geographic information in '.osm' format. This data includes elements such as buildings, roads, terrain, and vegetation. To bring this data into Blender, the Blender-OSM add-on can be used. This add-on allows users to import OSM data directly into Blender's 3D environment, with configurable options to include structures, terrain elevation, and even satellite imagery. Once imported, the scene is constructed based on real-world coordinates, and 3D models are automatically generated and positioned as shown in Figure 15. After the scene is loaded into Blender, users can apply materials, lighting, and camera setups using Blender's native tools. This imported outdoor environment then can be exported to USD format by Blender itself so that it can be loaded into Isaac sim. Then as explained in previous sections, dynamics and sensors can be implemented in this outdoor environment.

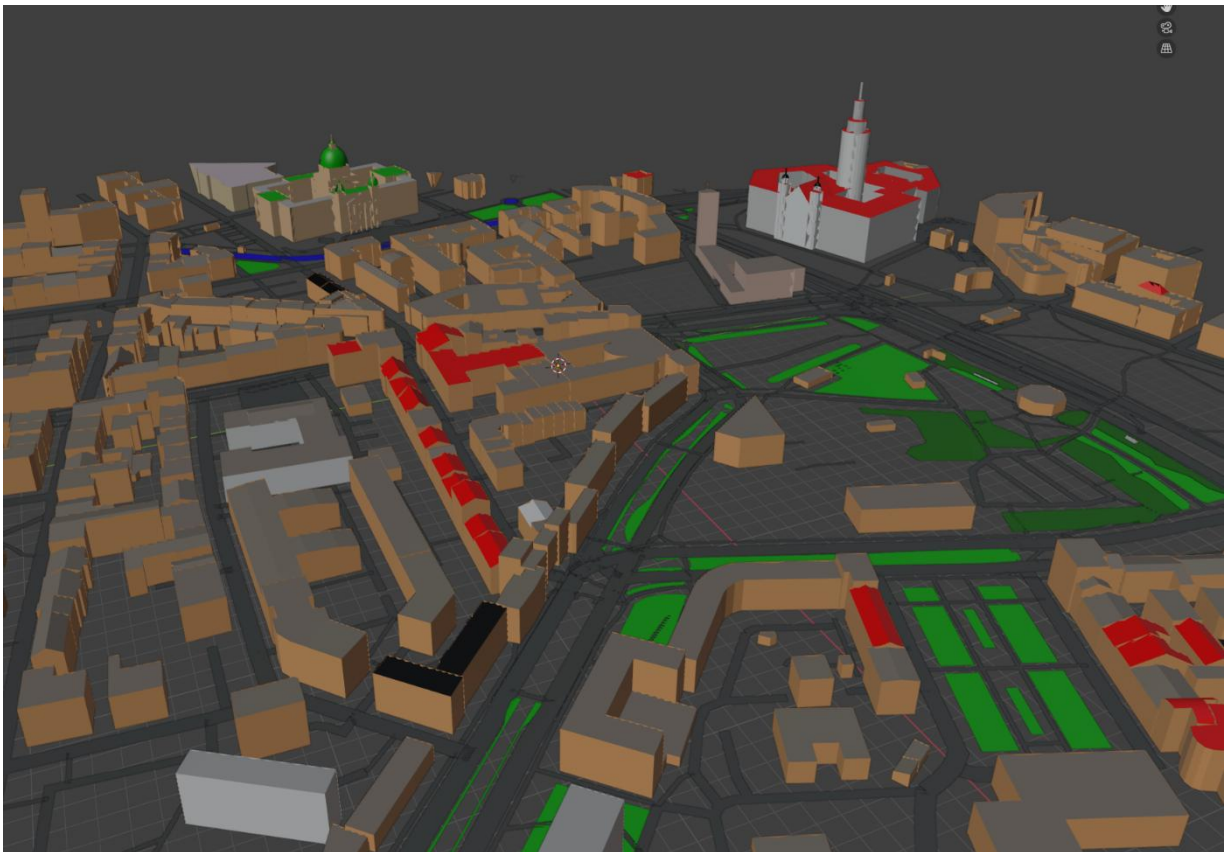


Figure 15: 3-D map of Oulu city imported to Blender using OSM

4.4 Dynamics detection using LiDAR ROS2 topics

Either in the simulation environment or the real environment, LiDAR sensor data can be published via ROS2 topics. ROS2 is an open-source framework that provides a robust and scalable foundation for building modular robotic applications. It is the successor to the original ROS, offering enhanced performance, security, and support for real-time systems. With ROS2, developers can leverage a rich set of tools and libraries to design software for a wide range of robots, from research platforms to industrial systems. The framework supports features such as inter-process communication, lifecycle management, multi-threading, and hardware abstraction, making it well-suited for complex and distributed robotics tasks. ROS 2 Humble release was used for this activity since it was well compatible with Isaac sim.

Once the LiDAR data is being published from Isaac sim, it can be visualized or processed in real time. The ROS2 topics that was published by Isaac Sim LiDAR sensor is available in all the ROS2 sourced terminals within the same network. Users can subscribe to those topics to process Lidar data. To detect the dynamics in the environment a program was developed using Python. Dynamic detection is done by the DBSCAN algorithm. This program subscribes to the LiDAR ROS2 topic output identified dynamic objects (in this case, dynamic objects are humans) on the terminal and in a CSV format as well. Following is the pseudo code of the program.

Start the program

Initialize parameters:

- output_interval (e.g., every 1 second)
- max_disappeared_frames (e.g., 100)
- threshold for dynamic point detection

Load static map for background removal

Create subscriber to LiDAR point cloud topic

Initialize:

- current time step
- human ID counter
- tracking dictionaries (tracks, last_seen)
- storage for frame results
- CSV file for output logging

On receiving a new LiDAR point cloud:

- Increment current time step
- Convert ROS point cloud to NumPy array
- Compare each point to static map to detect "dynamic" points
- If no dynamic points:
 - Mark missing tracks
 - Exit callback

- Cluster the dynamic points using DBSCAN to identify human-like clusters
- Compute centroids of each cluster
- If no centroids:
 - Mark missing tracks
 - Exit callback
- Assign or reassign IDs to centroids:
 - Compare each new centroid to existing tracks using Euclidean distance
 - If close enough to an existing track, reuse that ID
 - Otherwise, assign a new ID
 - Project centroid to 2D plane ($z=0$)
- Store results for the current frame
- Mark tracks as disappeared if not seen recently
- Periodically (based on `output_interval`):
 - Print tracking results
 - Write them to the CSV file

On shutdown:

- Close the CSV file
- Clean up the node

4.5 Emulating dynamics on Sionna and ray tracing

After the detection of human dynamics, the next task was to emulate those dynamics on top of a static ray tracing environment. NVIDIA Sionna ray tracing was used as the ray tracing utility in this activity. Usually in Sionna, the user can place transmitters and receivers in a loaded 3-D model of the physical environment which is known as a scene. This scene consists of everything that required for radio propagation and rendering. Then ray tracing can be done in the loaded scene. This 3-D model used in Sionna is a static model and to represent dynamics, candidate objects were required to be placed in the scene and change their locations with time as per the inputs given by dynamic detection algorithm.

4.5.1 Scene creation for Sionna

A Scene in Sionna RT represents a collection of objects, each defining geometry and material properties within a simulated environment. These objects, along with active elements like transmitters and receivers, form the basis for wireless communication simulations by enabling the calculation of propagation paths or CIR [22]. Scenes may also include cameras for rendering realistic visualizations. Scenes can be loaded from XML files using the `load_scene()` function, which parses and sets up all relevant components. These XML files can be created and edited in Blender using the Mitsuba-Blender add-on, following steps like those in Section 4.3, with additional provisions for defining candidate objects used to simulate dynamic elements like human movement. Radio materials in Sionna RT control interactions of electromagnetic waves

with surfaces, influencing reflection, transmission, and absorption, and are defined using parameters such as relative permittivity (ϵ_r) and conductivity (σ). All materials are assumed non-magnetic ($\mu_r = 1$). Sionna RT includes standard material models based on ITU guidelines [22] given in Figure 16.

Material name	Real part of relative permittivity		Conductivity [S/m]		Frequency range (GHz)
	a	b	c	d	
vacuum	1	0	0	0	0.001 – 100
itu_concrete	5.24	0	0.0462	0.7822	1 – 100
itu_brick	3.91	0	0.0238	0.16	1 – 40
itu_plasterboard	2.73	0	0.0085	0.9395	1 – 100
itu_wood	1.99	0	0.0047	1.0718	0.001 – 100
itu_glass	6.31	0	0.0036	1.3394	0.1 – 100
	5.79	0	0.0004	1.658	220 – 450
itu_ceiling_board	1.48	0	0.0011	1.0750	1 – 100
	1.52	0	0.0029	1.029	220 – 450
itu_chipboard	2.58	0	0.0217	0.7800	1 – 100
itu_plywood	2.71	0	0.33	0	1 – 40
itu_marble	7.074	0	0.0055	0.9262	1 – 60
itu_floorboard	3.66	0	0.0044	1.3515	50 – 100
itu_metal	1	0	10^7	0	1 – 100
itu_very_dry_ground	3	0	0.00015	2.52	1 – 10
itu_medium_dry_ground	15	-0.1	0.035	1.63	1 – 10
itu_wet_ground	30	-0.4	0.15	1.30	1 – 10

Figure 16: Radio material models in Sionna as per ITU-R P.2040-2 recommendation [22]

Sionna also supports defining custom radio materials by using the ‘RadioMaterial’ class as shown in Figure 17. Users can specify conductivity and relative permittivity. Also, the user can specify optional parameters such as cross-polarization discrimination coefficient, scattering pattern and scattering coefficient. These parameters are related to the diffuse scattering of electromagnetic waves.

```
custom_material = RadioMaterial("custom_material_01",
                                relative_permittivity=2.1,
                                conductivity=4.7,
                                scattering_coefficient=0.4,
                                xpd_coefficient=0.2,
                                scattering_pattern=LambertianPattern())
```

Figure 17: Defining a custom radio material in Sionna

Also, the user can define relative permittivity and conductivity as frequency dependent functions [23] as in Figure 18.

```
def compute_relative_permittivity(default_val,f_hz):
    # Simple example: permittivity drops with frequency
    return default_val - 1e-12 * f_hz

def compute_conductivity(default_val,f_hz):
    # Simple example: conductivity increases with frequency
    return default_val + 1e-12 * f_hz
```

Figure 18: Defining custom functions for frequency dependent ϵ_r and σ

Then the radio materials can be assigned to the respective scene objects via a callback function as in Figure 19.

```
def my_material_callback_floor(f_hz):
    relative_permittivity_floor = compute_relative_permittivity(8.0,f_hz)
    conductivity_floor = compute_conductivity(0.03,f_hz)
    return (relative_permittivity_floor, conductivity_floor)

custom_material_floor = RadioMaterial("my_material_floor",
                                     frequency_update_callback=my_material_callback_floor)
scene.add(custom_material_floor)

obj = scene.get("Plane") # obj is a SceneObject
obj.radio_material = "my_material_floor" # "obj" is made of "my_material_floor"
```

Figure 19: Assigning radio materials to scene objects via callback function

4.5.2 TX and RX configuration and placement

Once the scene is loaded in Sionna, the user can configure antenna arrays for all the transmitters and receivers with ‘scene.tx_array’ and ‘scene.rx_array’ APIs respectively. The arrays are typically specified using the ‘PlanarArray’ class, where users can define the number of rows and columns, the vertical and horizontal spacings between elements, the antenna radiation pattern, and the polarization scheme. This flexible setup enables simulation of a wide range of communication systems, from simple single-antenna setups to large multi-antenna configurations supporting advanced beamforming and MIMO techniques.

As in Figure 20, the transmitters are configured with an 8×2 planar array, spaced at 0.7 and 0.5 units vertically and horizontally, respectively. The antenna pattern follows the 3GPP TR 38.901 standard with dual vertical and horizontal polarizations ("VH"), modeling a realistic 5G base station. The receivers, on the other hand, are configured with a single-element planar array using a basic dipole radiation pattern and cross-polarization ("cross"), representing a typical mobile or IoT device. This setup reflects a massive MIMO transmission scenario towards simpler receivers.

```

# Configure antenna array for all transmitters
scene.tx_array = PlanarArray(num_rows=8,
                             num_cols=2,
                             vertical_spacing=0.5,
                             horizontal_spacing=0.5,
                             pattern="tr38901",
                             polarization="VH")

# Configure antenna array for all receivers
scene.rx_array = PlanarArray(num_rows=1,
                             num_cols=1,
                             vertical_spacing=0.5,
                             horizontal_spacing=0.5,
                             pattern="dipole",
                             polarization="cross")

```

Figure 20: Tx and Rx array configuration in Sionna

Then the user can define number of TX and RX and add them to the scene with the desired frequency. The Figure 21 shows two TXs and single RX setup in the scene with 120 GHz frequency.

```

scene.frequency = 120e9 # 120 GHz

# Create transmitter
tx1 = Transmitter(name="tx1",
                  position=[-6.0,7.8,1.2],
                  orientation=[0,0,0],
                  power_dbm=49)
scene.add(tx1)

tx2 = Transmitter(name="tx2",
                  position=[-7.0,-8.0,1.0],
                  orientation=[0,0,0],
                  power_dbm=33)
scene.add(tx2)

rx = Receiver(name="rx",
              position=[12.0,8.8,1.1],
              orientation=[0,0,0])
scene.add(rx)

# TX points towards RX
tx1.look_at(rx)
tx2.look_at(rx)

```

Figure 21 : Adding TX and RX to the scene

4.5.3 Mobility of the detected dynamic objects

To simulate the mobility of the dynamic objects in the environment (mobility of humans in this case), the user can change the position of the objects with time as given by the dynamic detection module. The Figure 22 shows an example of moving dynamic objects within the scene with time as given by the dynamic detection module output csv.

```
##### mobility of humans#####
import pandas as pd
import time # Simulate real-time updates

# Load CSV file
human_df = pd.read_csv("scene_dir/scene_1/dynamics/human_tracking.csv")

# Create human objects dynamically
humans = {i: scene.get(f"human_{i}") for i in range(1, 100)}

# Get unique time steps
time_steps = sorted(set(human_df["time"].unique()))

# Simulate movement over time
for t in time_steps:
    print(f"Updating positions for time step {t}")

    # Update human positions
    current_human_positions = human_df[human_df["time"] == t]
    for _, row in current_human_positions.iterrows():
        human_id = int(row["human_id"])
        if human_id in humans: # Ensure human exists
            new_position = [row["x"], row["y"], row["z"]]
            humans[human_id].position = new_position
            print(f"Human {human_id} moved to {new_position}")
```

Figure 22: Implementation of human mobility in the scene

4.5.4 Ray tracing and coverage map generation

Sionna RT can generate highly detailed wireless coverage maps. These maps are essential tools for visualizing signal strength, path loss, and overall network performance across complex 3D environments. By accurately modelling physical phenomena such as reflections, diffractions, and scattering, Sionna RT enables researchers and engineers to analyse how radio waves interact with real-world structures like buildings and terrain. Users can finely tune simulation parameters including map resolution, antenna beam patterns, number of rays, and ray interaction depth to match specific deployment scenarios and performance goals. The framework also supports advanced features like beamforming and multi-antenna configurations, making it highly

suitable for designing and optimizing modern communication systems such as 5G and beyond. Although 3D coverage maps are not natively rendered, users can generate 2D maps at various altitudes to approximate volumetric coverage, which is particularly useful for applications involving aerial vehicles or high-rise urban environments. The code snippet in Figure 23 shows the coverage map generation for both the transmitters that was added in the scene.

```
# Generate the coverage map for the current time step
cm = scene.coverage_map(
    num_samples=int(10e6),
    max_depth=5,
    cm_center=[0, 0, 0.3],
    cm_orientation=[0, 0, 0],
    cm_size=[50, 50],
    cm_cell_size=[2, 2]
)

# Render the coverage map
scene.render("cam", coverage_map=cm)
scene.render_to_file(camera="cam",
                    filename=f"scene_cm_{t}.png",
                    coverage_map=cm) # Render scene with coverage map to file
cm.show(tx=0); #2D visualization of the coverage map for 1st TX
cm.show(tx=1); #2D visualization of the coverage map for 2nd TX
```

Figure 23 : Coverage map generation for both the TXs in the scene

4.5.5 Access point selection based on received signal strength

The received signal strength (RSS) can be measured from ‘coverage_map.rss’ API and accordingly a decision can be taken for which Tx to be selected for the communication. The code snippet in Figure 24 shows a usage of RSS measurement for both the TXs. TX which is giving highest RSS is selected for the transmission.

```
# Define receiver logical cell position
rx_x = 15
rx_y = 11

# Extract RSS from both transmitters at the receiver's grid cell
rss_tx1 = cm.rss[0, rx_y, rx_x].numpy()
rss_tx2 = cm.rss[1, rx_y, rx_x].numpy()
```

Figure 24: RSS measurement from coverage map

4.6 NimbusRT based ray tracing on point clouds

NimbusRT[24] is an open-source ray tracing-based tool designed for radio channel characterization, utilizing point clouds as the intersection geometry. It computes paths consisting of specular reflections and diffractions between a set of transmitters and receivers. The main advantage is that ray tracing can be directly done on the sensor point clouds instead of artificially emulating dynamics on a static environment. Also, this can be integrated with Sionna RT so that the features available in Sionna can be utilized. But the radio materials should be assigned for the points based on the object. The code in Figure 25 demonstrates the usage of NimbusRT with Sionna for ray tracing on point clouds.

```

START

LOAD point cloud based scene

SET material properties:
    type = <material_type>
    scattering = <coefficient>
    pattern = directive(beamwidth = <value>)
APPLY to all material labels

CONFIGURE scene:
    frequency = <GHz>
    ray_tracing_params = {
        max_depth, enable_LOS, reflection, scattering, diffraction, RIS,
        threshold
    }

DEFINE antenna_array:
    layout = planar(rows, cols)
    spacing = (vertical, horizontal)
    pattern = <pattern>
    polarization = <V/H>

ASSIGN array to transmitter and receiver
ENABLE synthetic array

SET transmitter_position = [x1, y1, z1]
SET receiver_position = [x2, y2, z2]

COMPUTE coverage_map(resolution = [dx, dy])

END

```

Figure 25: NimbusRT and Sionna integration for ray tracing on point clouds.

5 RESULTS AND DISCUSSION

In this chapter, results of the implementation explained under the previous chapter will be reviewed and discussed.

5.1 Human dynamic detection

Initially static background was removed from the point cloud data for dynamic detection. Figures 26 and 27 show the point cloud data of the human dynamics before and after background removal respectively. The point cloud data after background removal was fed to the DBSCAN algorithm.

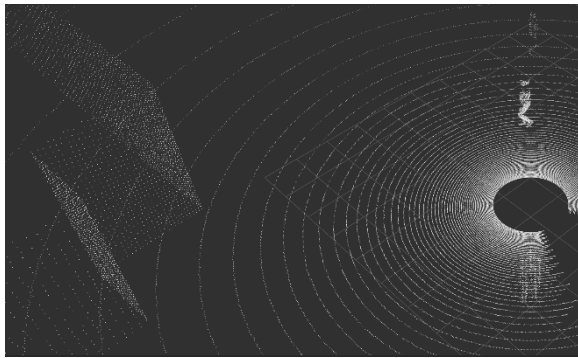


Figure 27: Before background removal

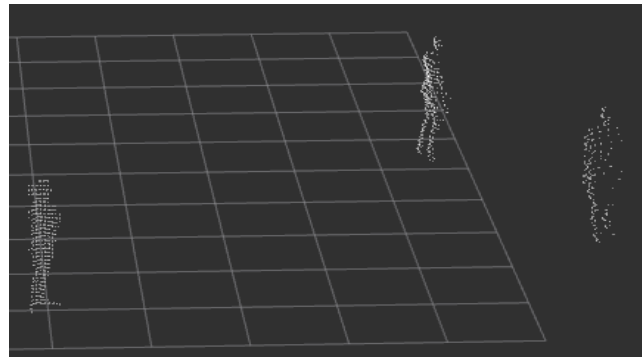


Figure 27: After background removal

When the background removed point cloud data is sent to the DBSCAN algorithm, it outputs human positioning with time. The Figure 28 shows the terminal output of the human dynamic detection program. It displays human positioning with time.

```
[Time 27]
human_1: x=-3.11, y=7.27, z=0.00
human_2: x=5.40, y=-9.28, z=0.00
human_3: x=-1.33, y=-2.42, z=0.00
human_4: x=-8.43, y=2.42, z=0.00

[Time 28]
human_1: x=-3.11, y=6.90, z=0.00
human_2: x=5.40, y=-9.29, z=0.00
human_3: x=-1.45, y=-2.03, z=0.00
human_4: x=-8.11, y=2.65, z=0.00

[Time 29]
human_1: x=-2.98, y=6.39, z=0.00
human_2: x=5.38, y=-9.27, z=0.00
human_3: x=-1.50, y=-1.56, z=0.00
human_4: x=-7.74, y=2.93, z=0.00
```

Figure 28 : Terminal output of human dynamic detection program

5.1.1 Accuracy check of human tracking

Once the human dynamic tracking part is completed, the accuracy of the algorithm was tested by defining a human walking path for a single human during the simulation level and then reconstructing the path based on the human tracking algorithm output. This was done for three trials with three different set of paths. The table 1 shows a set of planned and observed X, Y position coordinates of a single human in one trial.

Table 1: A set of planned and observed X, Y values of human position

Planned X	Planned Y	Observed X	Observed Y
5.30	12.33	5.27	12.27
-12.91	4.92	-12.80	4.91
-2.02	-1.95	-1.94	-1.77
8.24	5.08	8.08	4.95
0.17	-8.37	0.22	-8.40
3.81	-12.32	3.81	-12.10
2.01	8.29	2.00	8.26
-13.74	0.66	-13.67	0.72
-2.35	5.66	-2.40	5.42
-0.26	2.49	-0.14	2.50

Also, the figures 29, 30 and 31 show the planned and observed paths in X, Y plane for each test performed.

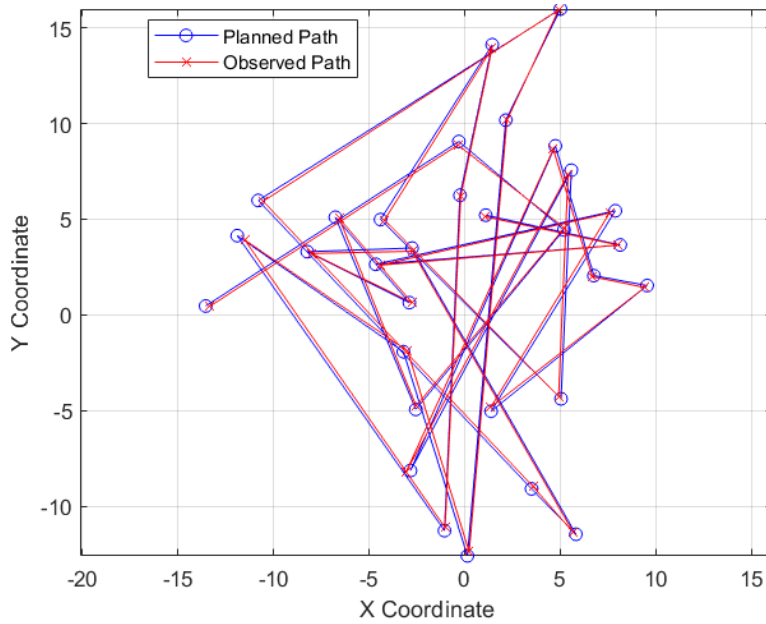


Figure 29: Planned and observed path for test 1

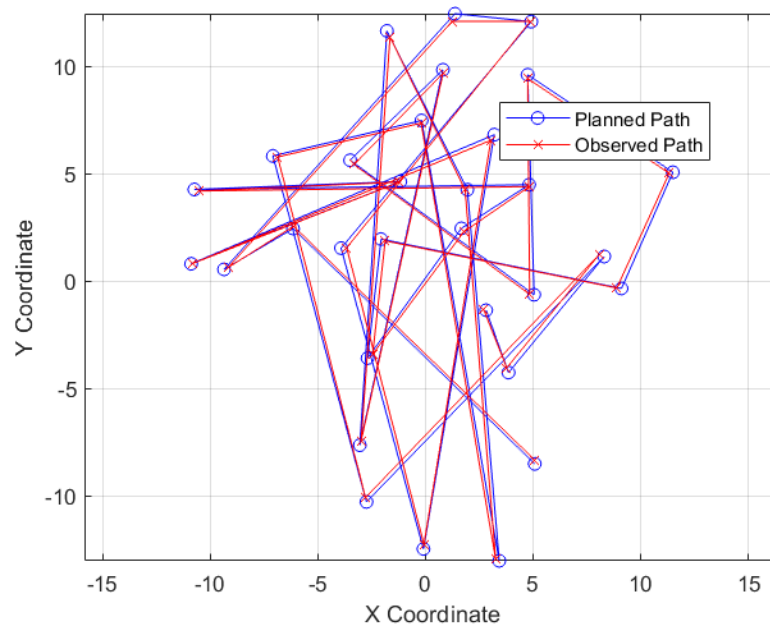


Figure 30: Planned and observed path for test 2

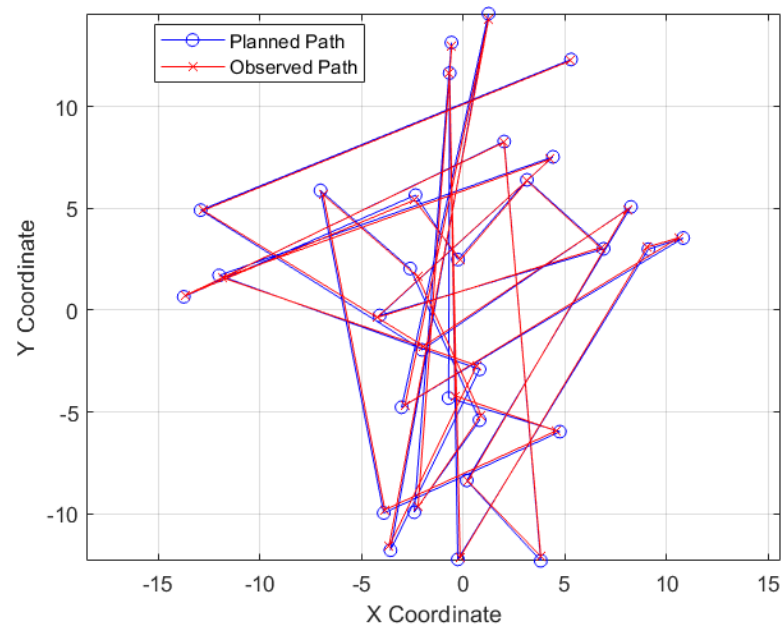


Figure 31: Planned and observed path for test 3

The table 2 shows the mean, max and standard deviation of position error values for each test performed.

Table 2: Error values for each test

Test Number	Mean Error	Max Error	Standard Deviation
1	0.1881	0.4632	0.0824
2	0.1961	0.3820	0.0789
3	0.1878	0.5374	0.1025

Across all tests, the mean error remains consistently low hovering around 0.19 which indicates a high level of accuracy in the predicted position of the human subject relative to their actual position. The maximum error values, while naturally higher, remain within acceptable limits, with the highest being 0.5374 in test 3. The standard deviation values, which reflect the variability in tracking accuracy, are also relatively low, demonstrating stable and reliable performance of the algorithm across different movement patterns and environments.

In addition to the quantitative results, the visual comparison of the planned and tracked paths further supports the effectiveness of the algorithm. In all three tests, the trajectory of the tracked human closely aligns with the intended movement path. This strong alignment suggests that the DBSCAN based approach is capable of correctly identifying and maintaining consistent clusters representing human positions, even in dynamic and potentially noisy environments.

Overall, the combination of low error metrics and high visual alignment provides robust evidence that the DBSCAN algorithm is effective in clustering and tracking individual humans from dynamic point clouds. These results confirm that the algorithm can be reliably applied in scenarios where accurate human tracking is critical.

5.2 Received Signal Strength variations

In this scenario, 2 transmitters were used (TX0, TX1). Each TX was 8x2 planar antenna array. Each element in the array was spaced 0.5 wavelength apart both vertically and horizontally. The array uses the standardized 3GPP TR 38.901 radiation pattern, ensuring realistic representation of antenna gain and beam shapes in 5G scenarios. Additionally, each element is dual polarized with vertical and horizontal polarization ("VH"), allowing the system to exploit polarization diversity and enhance signal robustness and throughput. This configuration is well-suited for evaluating performance in dense urban or high-throughput communication environments. Received Signal Strength was calculated for a specific location with different dynamics for each TX antenna. Also, access point selection criteria were introduced based on RSS.

5.2.1 RSS with different frequency settings.

RSS was measured for both the transmitters for different frequency values while keeping human dynamics in the environment constant (3 human dynamics in the scene) for each frequency.

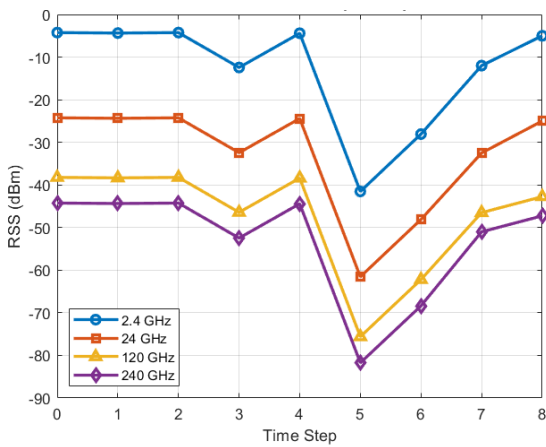


Figure 33 : TX0 RSS over time at multiple f

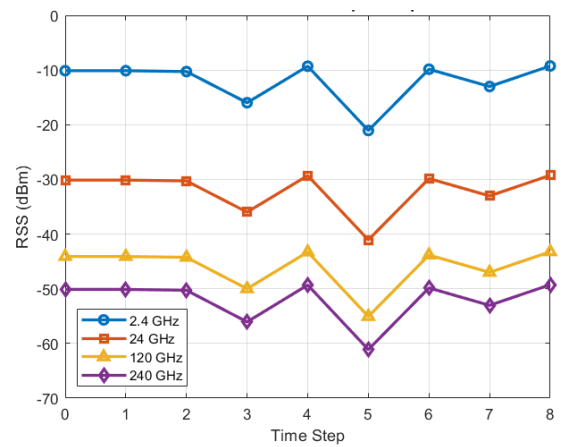


Figure 33 : TX1 RSS over time at multiple f

Figures 32 and 33 illustrate the RSS performance of TX0 and TX1 at four frequency bands (2.4 GHz, 24 GHz, 120 GHz, and 240 GHz) over time, with human dynamics held constant. As expected, RSS deteriorates with increasing frequency due to higher free-space path loss and greater susceptibility to human blockage. At 2.4 GHz, both transmitters maintain relatively high and stable RSS. In contrast, 120 GHz and 240 GHz show deeper fades and sharper RSS drops, particularly after time step 5 highlighting their vulnerability in dynamic environments.

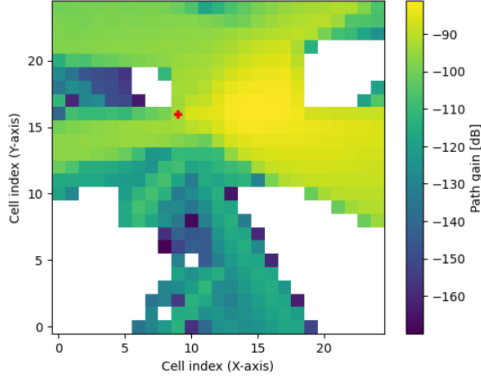


Figure 34: Path gain for TX0

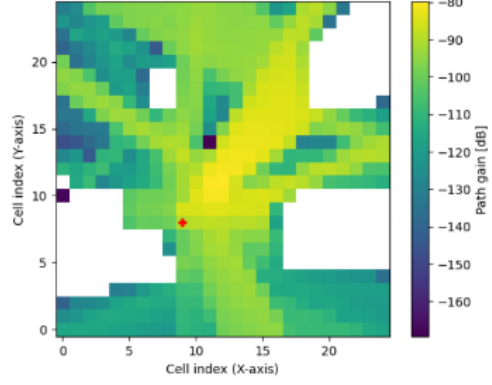


Figure 35: Path gain for TX1

Figures 34 and 35 provide spatial path gain maps for TX0 and TX1 at 120 GHz during time step 5. The coverage patterns reveal that Tx0 offers a broader high-gain region compared to TX1, though both maps exhibit significant attenuation zones caused by obstructions. These spatial variations confirm that frequency and transmitter location critically influence signal reachability and uniformity in human-populated scenarios.

5.2.2 RSS with different human dynamics

This experiment investigates the impact of human dynamics on received signal strength (RSS) at 120 GHz, a frequency relevant to future 6G systems. The testbed simulates a fixed wireless channel where two transmitters (TX0 and TX1) serve a receiver at a constant location while the number of moving human subjects varies from 1 to 10. For each human count scenario, the RSS from both transmitters is recorded over multiple time steps, capturing the effects of dynamic shadowing and multipath caused by human movement. This dataset enables quantitative analysis of how increasing human presence degrades signal quality, and it supports comparative evaluation of different transmission paths under the same channel conditions.

In addition to summarizing the average and minimum RSS values across human densities, the experiment also examines instantaneous RSS variations over time for each human scenario. By plotting the RSS for TX0 and TX1 across successive time steps, the analysis highlights how specific human movements cause transient signal fluctuations and fades. This time-resolved view of signal dynamics offers insights into the effects of human trajectories on link stability, including critical fade events and recovery patterns, particularly under dense human mobility environments.

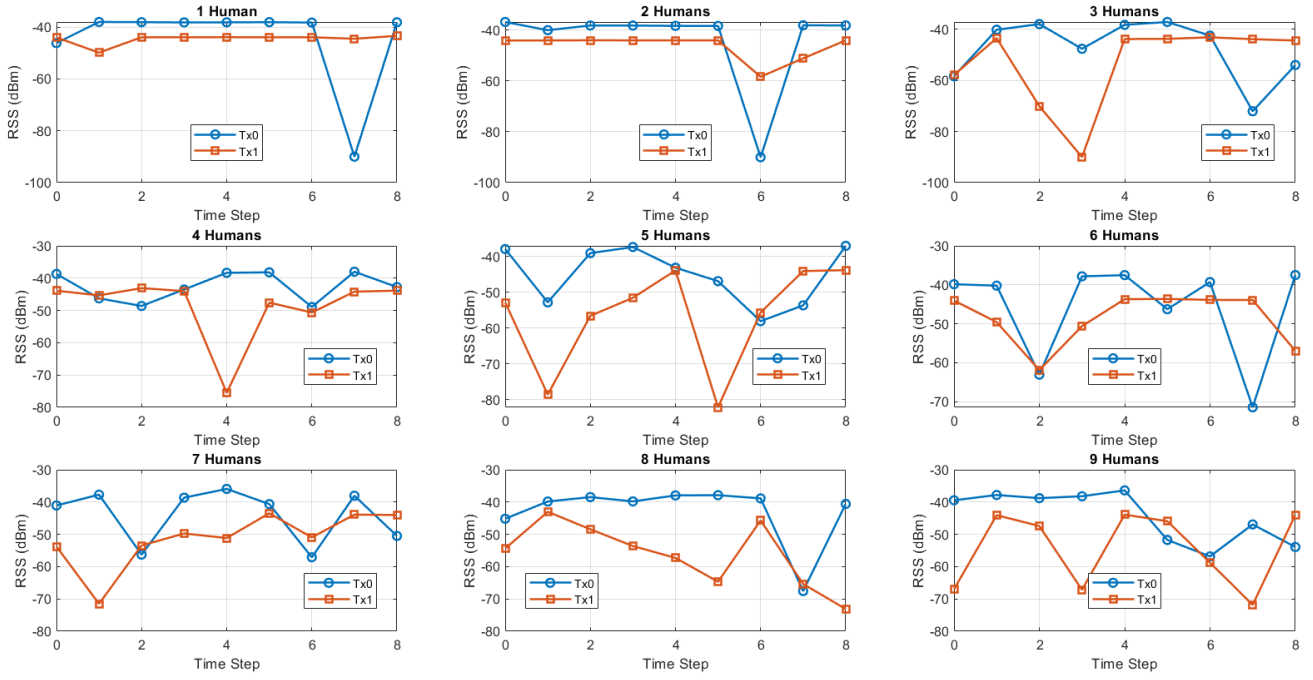


Figure 36: RSS over time for different human dynamics

Figure 36 shows the RSS over time for TX0 and TX1 at 120 GHz under varying numbers of human subjects. With fewer humans (1-2), the RSS remains relatively stable, with only occasional drops. As the number of humans increases, signal fluctuations become more frequent and severe, with some values falling below -80 dBm, particularly for TX0.

TX0 generally offers stronger RSS than TX1, but it also shows deeper fades likely due to direct line-of-sight blockages. In contrast, TX1, though weaker on average, exhibits more consistent behaviour. The 8-10 human scenarios highlight the increasing unpredictability and degradation of the wireless link due to dense human movement, emphasizing the need for robust beamforming or spatial diversity in such environments.

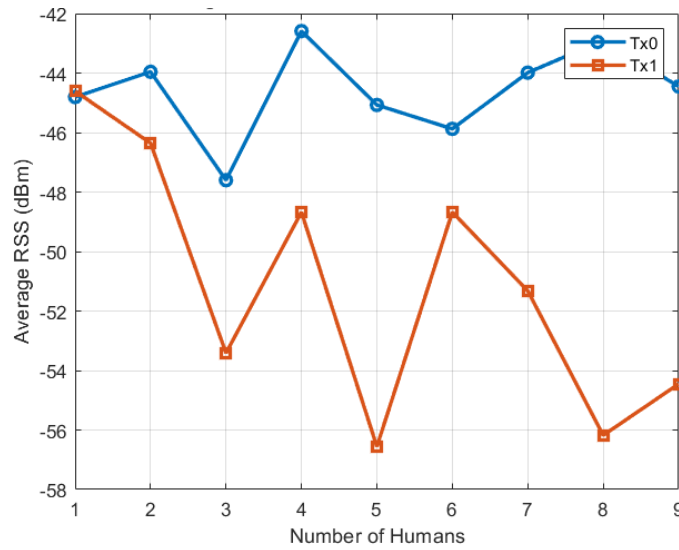


Figure 37: Average RSS vs number of humans

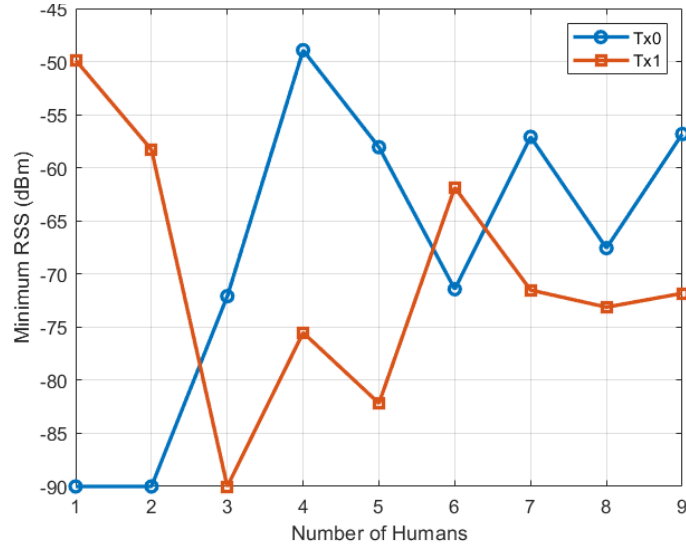


Figure 38: Minimum RSS vs number of humans

Figures 37 and 38 reveal how both the average and minimum RSS degrade with increasing human presence. While TX0 typically maintains higher average RSS than TX1, it also experiences greater variability and more severe minimum RSS drops, especially in lower human counts such as 3 humans. This suggests that although TX0 may benefit from a stronger line-of-sight path, it is more prone to sharp signal blockages.

TX1, though generally weaker, exhibits steadier minimum RSS across human densities, indicating more diffuse or indirect propagation paths. As human count increases, both transmitters show a trend of declining performance, reinforcing the impact of human induced fading and the need for link robustness in crowded environments.

5.2.3 *NimbusRT based ray tracing on point clouds*

In this method, Sionna ray tracing was performed at 120 GHz frequency on top of a point cloud data having human dynamics instead of a separate xml scene. Point cloud data should be given in PLY format for this scenario. Hence the LiDAR output ROS2 topic was processed to PLY format and Sionna ray tracing was performed. The Figure 38 shows a path gain map generated using this method.

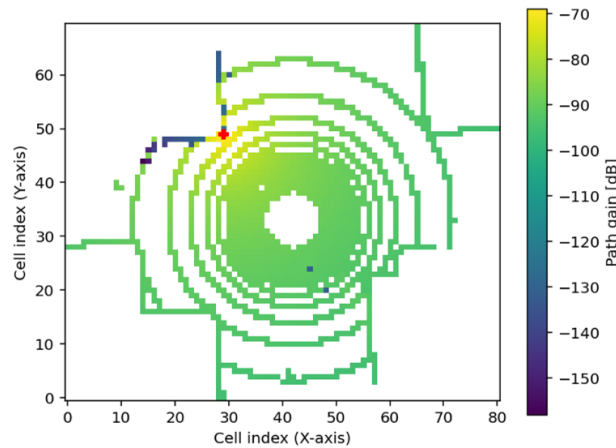


Figure 38: Path gain map generated using NimbusRT

By using the NimbusRT based ray tracing method on point clouds, RSS was plotted for different human dynamics as given in the Figure 39.

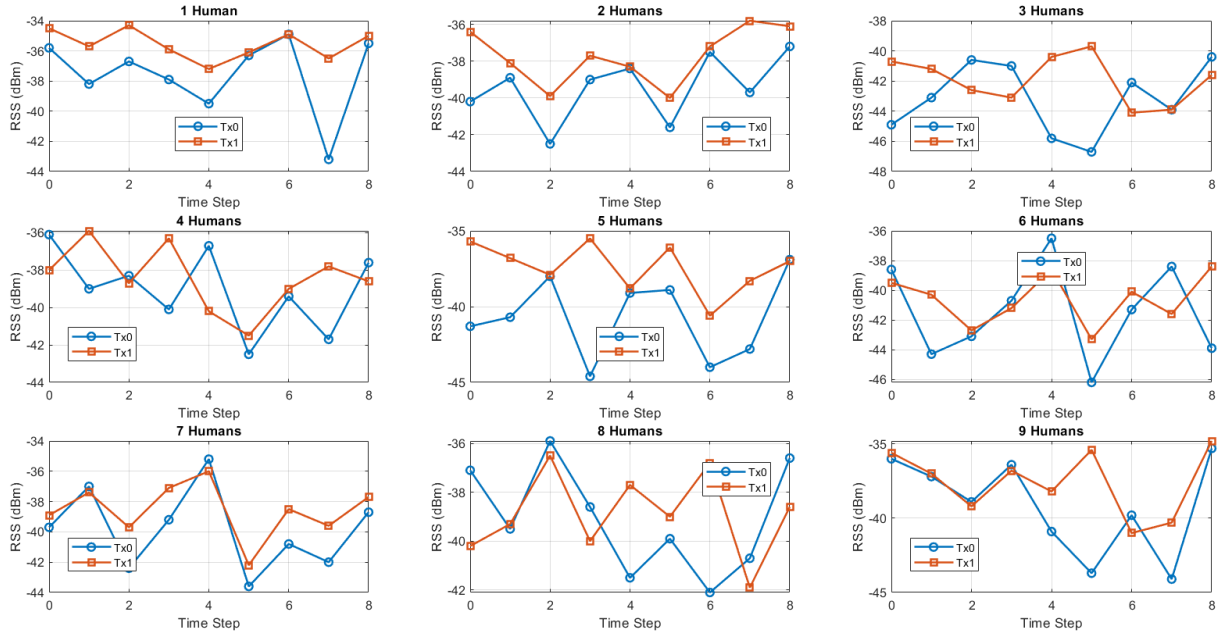


Figure 39: RSS over time for different human dynamics based on point clouds

Also, average and minimum RSS were plotted against number of humans in the method as well. The Figures 40 and 41 display those results

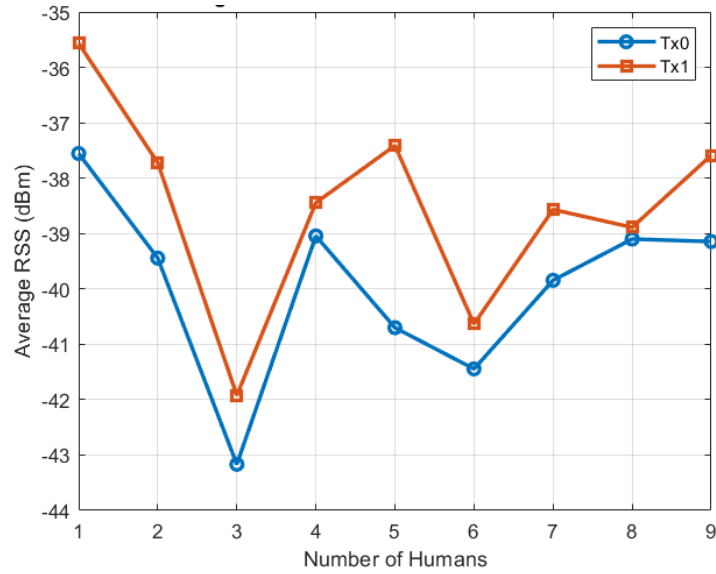


Figure 40: Average RSS vs number of humans based on point clouds

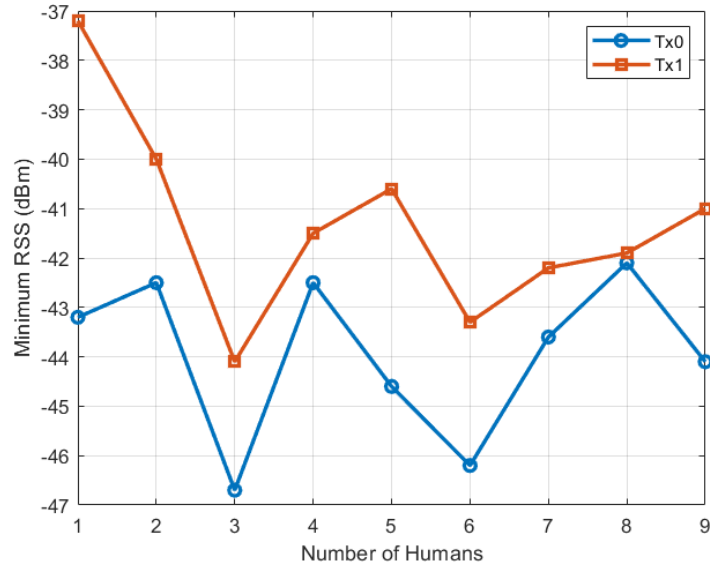


Figure 41: Minimum RSS vs number of humans based on point clouds

By comparing ray tracing results for dynamic object emulating on Sionna ray tracing (native SionnaRT approach) and NimbusRT based approaches, some differences can be observed in RSS. The main reason for this can be since point cloud-based approach may not have full geometric and material properties to trace all the paths as that in the native SionnaRT approach. But in this approach, dynamic object shape and orientation will be more accurate to that in real world scenario since point cloud will have the exact projection of the real-world dynamics. Compared to that, native SionnaRT approach will have candidate objects to represent the dynamics, and the geometries may not ideally match with that in the real-world dynamic environment.

5.2.4 Practical implementation in a physical environment using real sensors

The method discussed under the section 5.2.3 was practically implemented in multimodal sensing lab in the University of Oulu. Ouster OS1 lidar and Nerian Ruby 3D camera were used as per Figure 42 with ROS2 configurations in this case, and each sensor data was individually processed without fusing them as per the initial stage. During this implementation, path gain maps (coverage maps) were generated for each sensor.



Figure 42: LiDAR and camera setup

Then LiDAR point cloud was obtained as given in the Figure 43

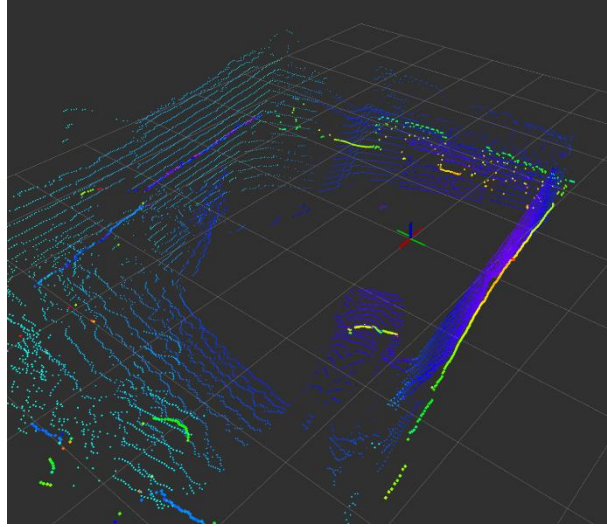


Figure 43: Ouster OS1 Lidar point cloud

By performing ray tracing on Lidar point cloud, path gain map was generated as in Figure 44.

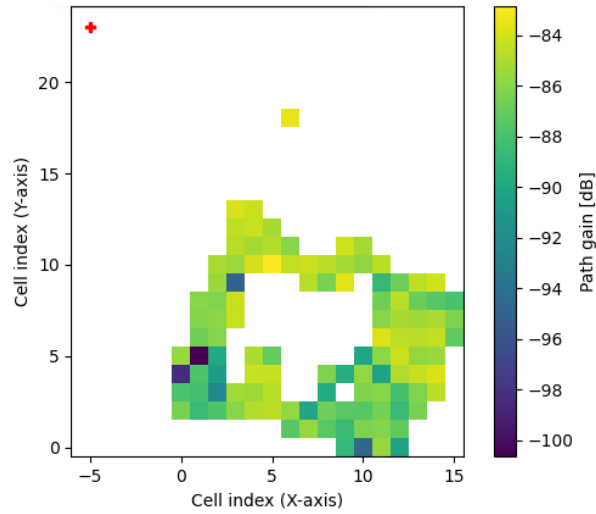


Figure 44: Path gain map generated by ray tracing on Lidar PC

The camera is capturing an angular portion of the space only and not a 360^0 of view. The camera was configured to publish 3-D point cloud topic with 'rgb' information as displayed in the Figure 45.

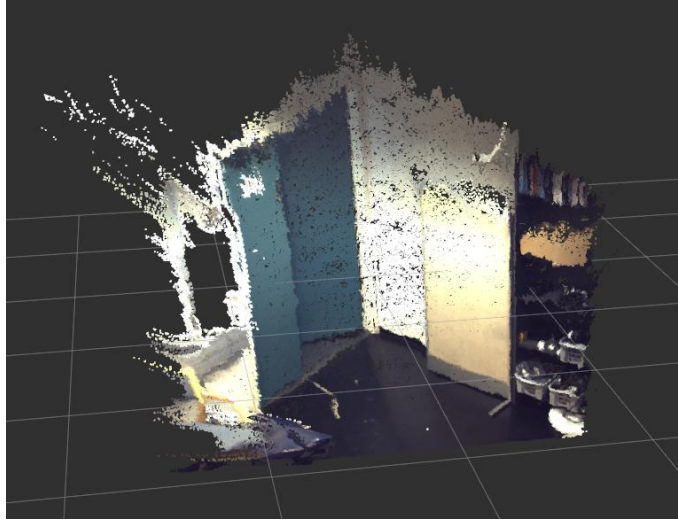


Figure 45: Camera 3-D point cloud

By using the camera point cloud data, ray tracing was done, and path gain map was generated as per the Figure 46. This does not contain coverage map for the entire room since the camera is capturing only a portion of the room.

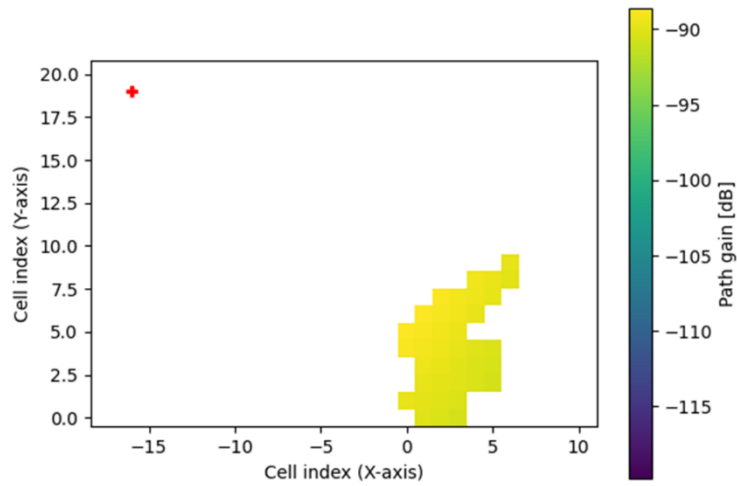


Figure 46: Path gain map generated by ray tracing on camera PC

In future work, multiple cameras can be used to cover the full environment and LiDAR, and camera point clouds can be merged [9]. Also, dynamics can be introduced. By using universal software radio peripheral (USRP) devices, real channel measurements can be obtained. Then that can be compared with the digital twin-based measurements for the accuracy check of the DT.

6 SUMMARY

This research explains the importance and the usage of dynamic digital twins in the context of 6G and how to replicate different levels of the network in improving the wireless network performance especially in enabling AI-driven enhancements. Practical and well robust wireless systems can be introduced with the use of digital twins especially for highly dynamic environments where the traditional approaches may not be accurate enough. This thesis explains each section and their implementation of a dynamic digital twin.

There is a review of some significant prior work that has been considered during the implementation of DT together with the theoretical background. Also, this contains information regarding necessary powerful tools such as NVIDIA Isaac Sim, NVIDIA SionnaRT and Blender and how to use them through the roadmap of a DT.

The implementation of a digital twin has been divided into several subcategories, and each category is well explained with the related tool usages, pseudocodes and scripting and machine learning usages whenever required. This drives from dynamic extraction using sensor data, emulating dynamics on a ray tracing environment and observing received signal strength variations due to dynamics. This observed data can be fed back to main control units so that dynamic digital twin can help to optimize the network.

By observing the results, it can be clearly identifying the impact of dynamics which may create blockages and hence will affect channel conditions especially at higher frequencies. Also, it helps to analyze each access point status individually so that the user can generate individual results per access point. These qualitative and quantitative results will help AI-driven network control units to predict the blockages and optimally use the network resources so that overall network performance can be increased.

The main sensor that was used here was LiDAR. In future, this research work can be further extended to use multi-model sensing by including camera information as well. LiDAR will be suitable for capturing depth information and the camera will help to provide visually rich rgb point cloud. Cameras capture high-resolution visual details, including colors and textures that LiDAR cannot detect. By merging these sensor data will allow generating precise 3D spatial data of the environment.

7 REFERENCES

- [1] M. Shafi, R. K. Jha and S. Jain, "6G: Technology Evolution in Future Wireless Networks," in *IEEE Access*, vol. 12, pp. 57548-57573, 2024
- [2] H. Pennanen, T. Hänninen, O. Tervo, A. Tölli and M. Latva-Aho, "6G: The Intelligent Network of Everything," in *IEEE Access*, vol. 13, pp. 1319-1421, 2025.
- [3] X. Lin, L. Kundu, C. Dick, E. Obiodu, T. Mostak and M. Flaxman, "6G Digital Twin Networks: From Theory to Practice," in *IEEE Communications Magazine*, vol. 61, no. 11, pp. 72-78, November 2023.
- [4] D. Marasinghe, N. Rajatheva and M. Latva-aho, "LiDAR Aided Human Blockage Prediction for 6G," 2021 IEEE Globecom Workshops (GC Wkshps), Madrid, Spain, 2021.
- [5] V. C. Andrei et al., "Demonstration of a Continuously Updated, Radio-Compatible Digital Twin for Robotic Integrated Sensing and Communications," 2025 IEEE 5th International Symposium on Joint Communications & Sensing (JC&S), Oulu, Finland, 2025, pp. 1-2.
- [6] A. Alkhateeb, S. Jiang and G. Charan, "Real-Time Digital Twins: Vision and Research Directions for 6G and Beyond," in *IEEE Communications Magazine*, vol. 61, no. 11, pp. 128-134, November 2023.
- [7] J. Hoydis et al., "Sionna RT: Differentiable Ray Tracing for Radio Propagation Modeling," 2023 IEEE Globecom Workshops (GC Wkshps), Kuala Lumpur, Malaysia, 2023, pp. 317-321.
- [8] C. Li, Y. Ren and B. Liu, "PCGen: Point Cloud Generator for LiDAR Simulation," 2023 IEEE International Conference on Robotics and Automation (ICRA), London, United Kingdom, 2023, pp. 11676-11682
- [9] S. Bae, D. Han and S. Park, "A New Approach to Lidar and Camera Fusion for Autonomous Driving," 2023 International Conference on Artificial Intelligence in Information and Communication (ICAIIIC), Bali, Indonesia, 2023, pp. 751-753
- [10] J. Fang et al., "LiDAR-CS Dataset: LiDAR Point Cloud Dataset with Cross-Sensors for 3D Object Detection," 2024 IEEE International Conference on Robotics and Automation (ICRA), Yokohama, Japan, 2024, pp. 14822-14829
- [11] R. Ranjith, J. J. Athanesious and V. Vaidehi, "Anomaly detection using DBSCAN clustering technique for traffic video surveillance," 2015 *Seventh International Conference on Advanced Computing (ICoAC)*, Chennai, India, 2015, pp. 1-6
- [12] M. E. Yabroudi, K. Awedat, R. C. Chabaan, O. Abudayyeh and I. Abdel-Qader, "Adaptive DBSCAN LiDAR Point Cloud Clustering For Autonomous Driving Applications," 2022 *IEEE International Conference on Electro Information Technology (eIT)*, Mankato, MN, USA, 2022, pp. 221-224

- [13] M. Jacinto *et al.*, "Pegasus Simulator: An Isaac Sim Framework for Multiple Aerial Vehicles Simulation," *2024 International Conference on Unmanned Aircraft Systems (ICUAS)*, Chania - Crete, Greece, 2024, pp. 917-922
- [14] Z. Yun and M. F. Iskander, "Ray Tracing for Radio Propagation Modeling: Principles and Applications," in *IEEE Access*, vol. 3, pp. 1089-1100, 2015
- [15] J. Pyhtilae, J. Kokkonen, P. Sangi, N. Vaara and M. Juntti, "Ray Tracing Based Radio Channel Modelling Applied to RIS," *WSA & SCC 2023; 26th International ITG Workshop on Smart Antennas and 13th Conference on Systems, Communications, and Coding*, Braunschweig, Germany, 2023, pp. 1-6
- [16] N. Jayaweera *et al.*, "LiDAR aided Wireless Networks - LoS Detection and Prediction based on Static Maps," *2022 IEEE 96th Vehicular Technology Conference (VTC2022-Fall)*, London, United Kingdom, 2022, pp. 1-6
- [17] N. Vaara, P. Sangi, M. B. López and J. Heikkilä, "A Ray Launching Approach for Computing Exact Paths with Point Clouds," *2024 IEEE International Conference on Acoustics, Speech, and Signal Processing Workshops (ICASSPW)*, Seoul, Korea, Republic of, 2024, pp. 800-804
- [18] Alatalo T, Koskela T, Pouke M, Alavesä P & Ojala T (2016), "VirtualOulu: Collaborative, immersive and extensible 3D city model on the web" Proc. 21st Annual International Conference on 3D Web Technology (Web3D), Anaheim, CA, USA, 95-103
- [19] Ouster OS0-128 Rev 7 Lidar sensor. Accessed 22 April 2025. [Online]. Available: <https://www.general-laser.at/en/shop-en/ouster-os0-128-lidar-sensor-en>
- [20] Oulu City view in Open Street Map. Accessed 22 April 2025. [Online]. Available: <https://www.openstreetmap.org>
- [21] isaac_tutorial_ros2_rtx_lidar_graph_rviz.png. Accessed 22 April 2025. [Online]. Available: https://docs.omniverse.nvidia.com/isaacsim/latest/_images/isaac_tutorial_ros2_rtx_lidar_graph_rviz.png
- [22] Sionna Ray Tracing official documentation. Accessed 23 April 2025. [Online]. Available: <https://nvlabs.github.io/sionna/rt/index.html>
- [23] N. Nasir and M. Al Ahmad, "Cells Electrical Characterization: Dielectric Properties, Mixture, and Modeling Theories," *Journal of Engineering*, vol. 2020
- [24] NimbusRT. Accessed 31 March 2025. [Online]. Available: <https://github.com/nvaara/NimbusRT>